

Python Strings are Arrays. They are arrays of bytes representing unicode characters. We can assign a string literal enclosed in a single quotation ('Udacity') or double quotation ("Udacity") marks to a variable.

str1 = "Udacity" Common String Methods Let us see some common methods that we use on string variables. Refer here for more

Exercise 1. Reverse Strings In this first exercise, the goal is to write a function that takes a string as input and then returns the reversed string.

For example, if the input is the string "water", then the output should be "retaw".

While you're working on the function and trying to figure out how to manipulate the string, it may help to use the print statement so you can see the effects of whatever you're trying.

Note - You can use built-in method len() on the string.

```
#solution 1
def string_reverser(our_string):
    result = []
    last_index = len(our_string) - 1
    while last_index >= 0:
        result.append(our_string[last_index])
        last_index -= 1
    return "".join(result)

# Test Cases

print ("Pass" if ('retaw' == string_reverser('water')) else "Fail")
print ("Pass" if ('!noitalupinam gnirts gnicitcarP' ==
string_reverser('Practicing string manipulation!')) else "Fail")
print ("Pass" if ('3432 :si edoc esuoh ehT' == string_reverser('The
house code is: 2343')) else "Fail")

Pass
Pass
Pass

def string_reverser(our_string):
    return our_string[::-1]

# Test Cases

print ("Pass" if ('retaw' == string_reverser('water')) else "Fail")
print ("Pass" if ('!noitalupinam gnirts gnicitcarP' ==
string_reverser('Practicing string manipulation!')) else "Fail")
print ("Pass" if ('3432 :si edoc esuoh ehT' == string_reverser('The
house code is: 2343')) else "Fail")
```

Pass
Pass
Pass

Exercise 2. Anagrams The goal of this exercise is to write some code to determine if two strings are anagrams of each other.

An anagram is a word (or phrase) that is formed by rearranging the letters of another word (or phrase).

For example:

"rat" is an anagram of "art" "alert" is an anagram of "alter" "Slot machines" is an anagram of "Cash lost in me" Your function should take two strings as input and return True if the two words are anagrams and False if they are not.

You can assume the following about the input strings:

No punctuation No numbers No special characters Note - You can use built-in methods len(), lower() and sort() on strings.

```
def anagram_checker(str1, str2):  
    # Remove spaces and convert to lowercase  
    str1_cleaned = str1.replace(" ", "").lower()  
    str2_cleaned = str2.replace(" ", "").lower()  
  
    # Sort and compare  
    return sorted(str1_cleaned) == sorted(str2_cleaned)  
  
# Test Cases  
print ("Pass" if anagram_checker('rat','art') else "Fail")  
print ("Pass" if not (anagram_checker('water','waiter')) else "Fail")  
print ("Pass" if anagram_checker('Dormitory','Dirty room') else  
"Fail")  
print ("Pass" if anagram_checker('Slot machines', 'Cash lost in me')  
else "Fail")  
print ("Pass" if not (anagram_checker('A gentleman','Elegant men'))  
else "Fail")  
print ("Pass" if anagram_checker('Time and tide wait for no  
man','Notified madman into water') else "Fail")
```

Pass
Pass
Pass
Pass
Pass
Pass
Pass

Exercise 3. Reverse the words in sentence Given a sentence, reverse each word in the sentence while keeping the order the same!

Solution 1

```
def word_flipper(our_string):  
    """  
    Flip the individual words in a sentence  
  
    Args:  
        our_string(string): Strings to have individual words flip  
    Returns:  
        string: String with words flipped  
    """  
  
    word_list = our_string.split(" ")  
  
    for idx in range(len(word_list)):  
        word_list[idx] = word_list[idx][::-1]  
  
    return " ".join(word_list)  
  
print ("Pass" if ('retaw' == word_flipper('water')) else "Fail")  
print ("Pass" if ('sihT si na elpmaxe' == word_flipper('This is an  
example')) else "Fail")  
print ("Pass" if ('sihT si eno llams pets rof ...' ==  
word_flipper('This is one small step for ...')) else "Fail")  
  
Pass  
Pass  
Pass  
  
def word_flipper(our_string):  
    # Split sentence into words  
    words = our_string.split()  
    # Reverse each word and join them back with space  
    flipped = [word[::-1] for word in words]  
    return ' '.join(flipped)  
  
print ("Pass" if ('retaw' == word_flipper('water')) else "Fail")  
print ("Pass" if ('sihT si na elpmaxe' == word_flipper('This is an  
example')) else "Fail")  
print ("Pass" if ('sihT si eno llams pets rof ...' ==  
word_flipper('This is one small step for ...')) else "Fail")  
  
Pass  
Pass  
Pass
```

Exercise 4. Hamming Distance In information theory, the Hamming distance between two strings of equal length is the number of positions at which the corresponding symbols are different. Calculate the Hamming distance for the following test cases.

Solution

```
def hamming_distance(str1, str2):  
    """  
    Calculate the hamming distance of the two strings  
  
    Args:  
        str1(string),str2(string): Strings to be used for finding the  
        hamming distance  
    Returns:  
        int: Hamming Distance  
    """  
  
    if len(str1) == len(str2):  
        count = 0  
  
        for char in range(len(str1)):  
            if str1[char] != str2[char]:  
                count+=1  
  
        return count  
  
    return None  
  
print ("Pass" if (10 == hamming_distance('ACTTGACCGGG','GATCCGGTACA'))  
else "Fail")  
print ("Pass" if (1 == hamming_distance('shove','stove')) else  
"Fail")  
print ("Pass" if (None == hamming_distance('Slot machines', 'Cash  
lost in me')) else "Fail")  
print ("Pass" if (9 == hamming_distance('A gentleman','Elegant men'))  
else "Fail")  
print ("Pass" if (2 ==  
hamming_distance('0101010100011101','0101010100010001')) else "Fail")  
  
Pass  
Pass  
Pass  
Pass  
Pass
```

Reversing a linked list exercise Given a singly linked list, return another linked list that is the reverse of the first.

Helper Code

```
class Node:  
    def __init__(self, value):  
        self.value = value
```

```

        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, value):
        if self.head is None:
            self.head = Node(value)
            return

        node = self.head
        while node.next:
            node = node.next

        node.next = Node(value)

    def __iter__(self):
        node = self.head
        while node:
            yield node.value
            node = node.next

    def __repr__(self):
        return str([v for v in self])

def reverse(linked_list):
    new_list = LinkedList()
    prev_node = None
    for value in linked_list:
        new_node = Node(value)
        new_node.next = prev_node
        prev_node = new_node
    new_list.head = prev_node
    return new_list

l1 = LinkedList()
for value in [4,2,5,1,-3,0]:
    l1.append(value)

flipped = reverse(l1)
is_correct = list(flipped) == list([0,-3,1,5,2,4]) and list(l1) == list(reverse(flipped))
print("Pass" if is_correct else "Fail")

Pass

import matplotlib.pyplot as plt

# --- Node Definition ---
class Node:

```

```

def __init__(self, value):
    self.value = value
    self.next = None

# --- Reversal Logic with Step Capture ---
def reverse_and_capture_steps(values):
    steps = []
    prev_node = None

    for value in values:
        new_node = Node(value)
        new_node.next = prev_node
        prev_node = new_node

        # Capture snapshot
        current = new_node
        snapshot = {
            "nodes": [],
            "new_node": new_node.value,
            "prev_node": new_node.next.value if new_node.next else
None
        }

        while current:
            snapshot["nodes"].append(current.value)
            current = current.next

        steps.append(snapshot)

    return steps

# --- Plotting Function ---
def plot_steps_with_pointers(steps):
    n_steps = len(steps)
    fig_height = max(4, n_steps * 2)
    fig, axes = plt.subplots(n_steps, 1, figsize=(12, fig_height))

    if n_steps == 1:
        axes = [axes]

    for i, (ax, step) in enumerate(zip(axes, steps)):
        nodes = step["nodes"]
        new_node = step["new_node"]
        prev_node = step["prev_node"]

        ax.clear()
        ax.set_xlim(-1, len(nodes) * 2)
        ax.set_ylim(-2, 2)
        ax.axis("off")

```

```

# Draw nodes
for j, value in enumerate(nodes):
    x = j * 2
    y = 0

    color = "#a2d5c6" if value == new_node else "#74b9ff"
    edgecolor = "#0984e3" if value == new_node else "#0984e3"
    ax.text(x, y, str(value),
            fontsize=14,
            bbox=dict(boxstyle="round,pad=0.4",
facecolor=color, edgecolor=edgecolor),
            ha='center', va='center')

    # Draw arrow to next node if not last
    if j < len(nodes) - 1:
        ax.annotate("",
                    xy=(x + 1.6, y),
                    xytext=(x + 0.4, y),
                    arrowprops=dict(arrowstyle="->",
color="#636e72", lw=2))

    # Annotate new_node label above
    if new_node is not None:
        new_index = nodes.index(new_node)
        ax.annotate("new_node",
                    xy=(new_index * 2, 0.8),
                    xytext=(new_index * 2, 1.5),
                    ha='center',
                    fontsize=10,
                    color="#00b894",
                    arrowprops=dict(arrowstyle="->",
color="#00b894", lw=2))

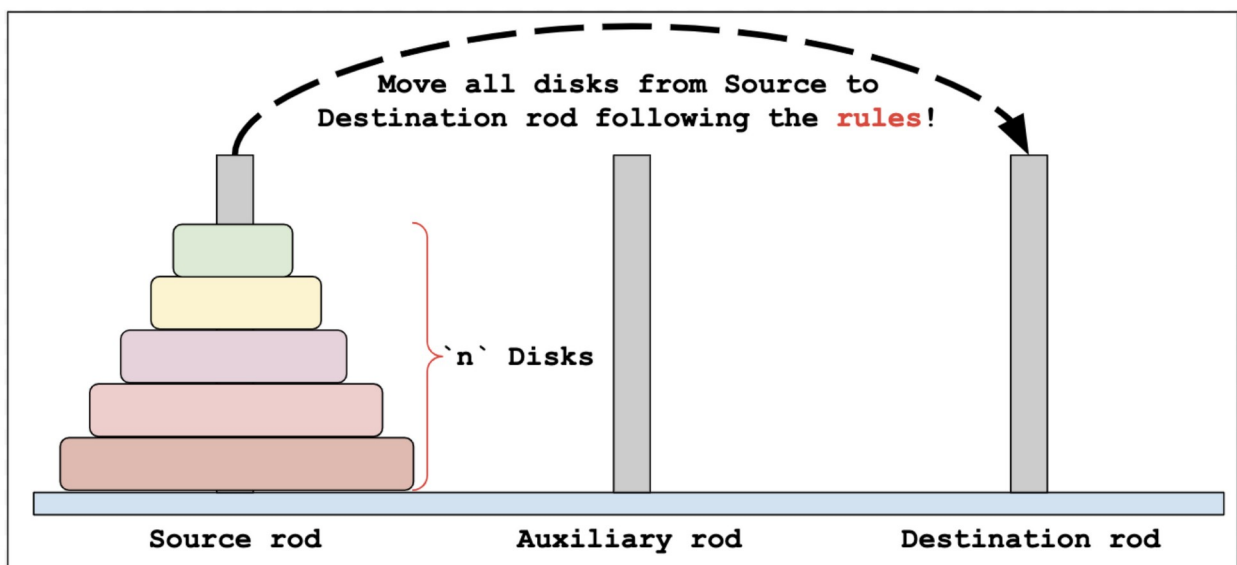
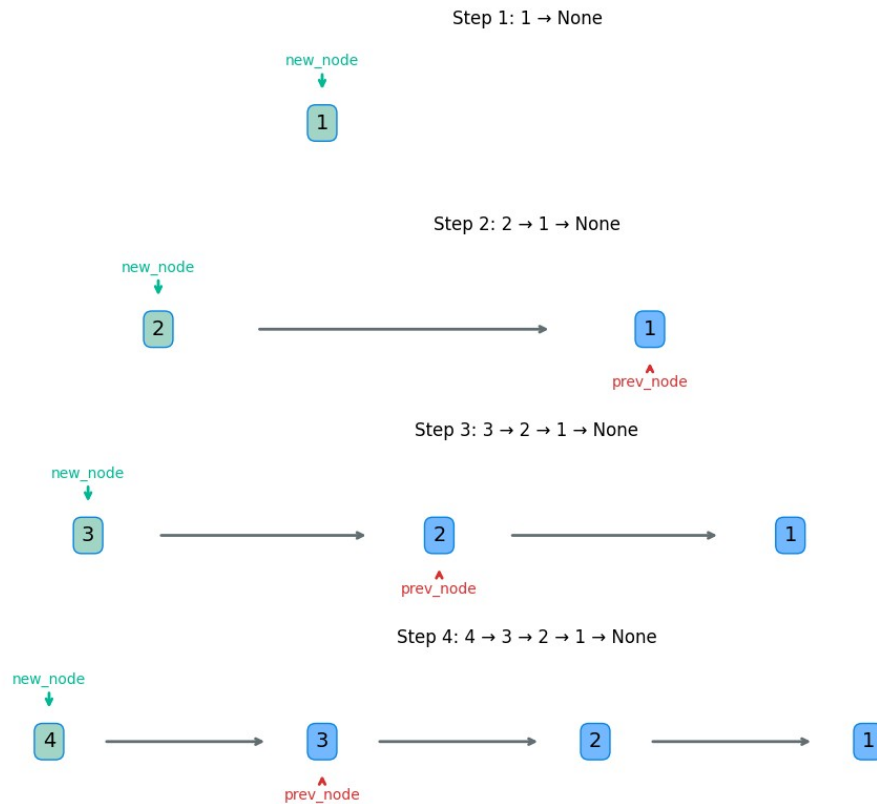
    # Annotate prev_node label below
    if prev_node is not None and prev_node in nodes:
        prev_index = nodes.index(prev_node)
        ax.annotate("prev_node",
                    xy=(prev_index * 2, -0.8),
                    xytext=(prev_index * 2, -1.5),
                    ha='center',
                    fontsize=10,
                    color="#d63031",
                    arrowprops=dict(arrowstyle="->",
color="#d63031", lw=2))

    ax.set_title(f"Step {i+1}: {' → '.join(map(str, nodes))} →
None", fontsize=12, pad=15)

plt.tight_layout()
plt.show()

```

```
# --- Run Example ---
if __name__ == "__main__":
    values = [1, 2, 3, 4]
    steps = reverse_and_capture_steps(values)
    plot_steps_with_pointers(steps)
```




```

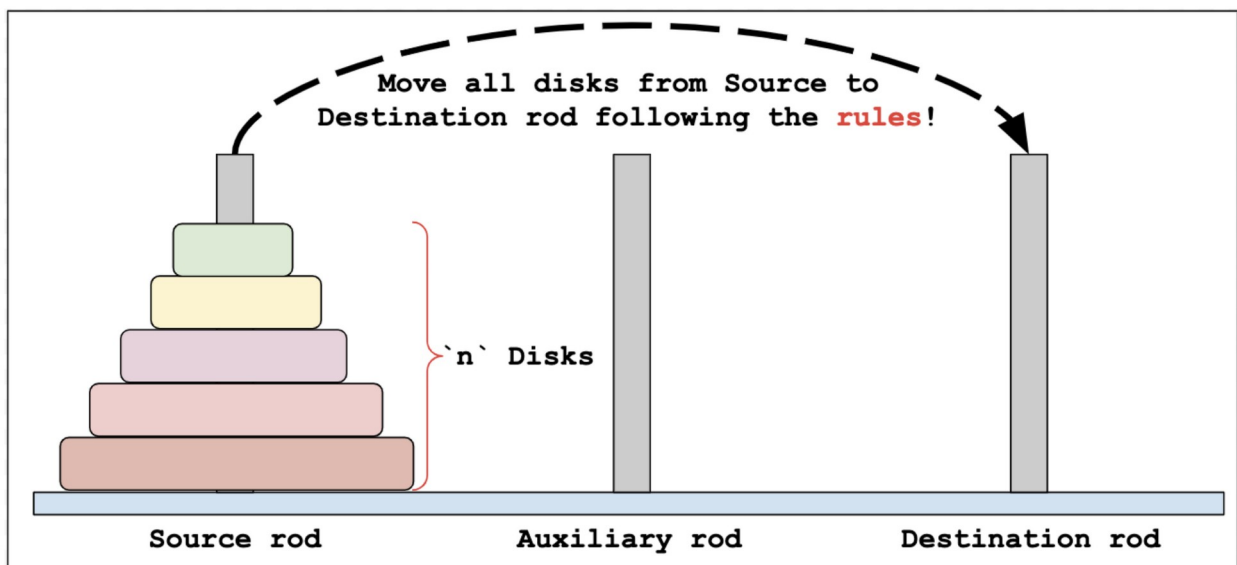
#solution 2 in place reverse.
# Interview Tip:
#“I just track where I came from, where I am, and where I’m going –
and flip the pointer at each step.”
def reverse(linked_list):
    prev_node = None
    current_node = linked_list.head

    while current_node is not None:
        next_node = current_node.next    # Save next node
        current_node.next = prev_node    # Reverse pointer
        prev_node = current_node         # Move prev_node forward
        current_node = next_node         # Move current_node forward

    linked_list.head = prev_node        # Update head to new front
    return linked_list

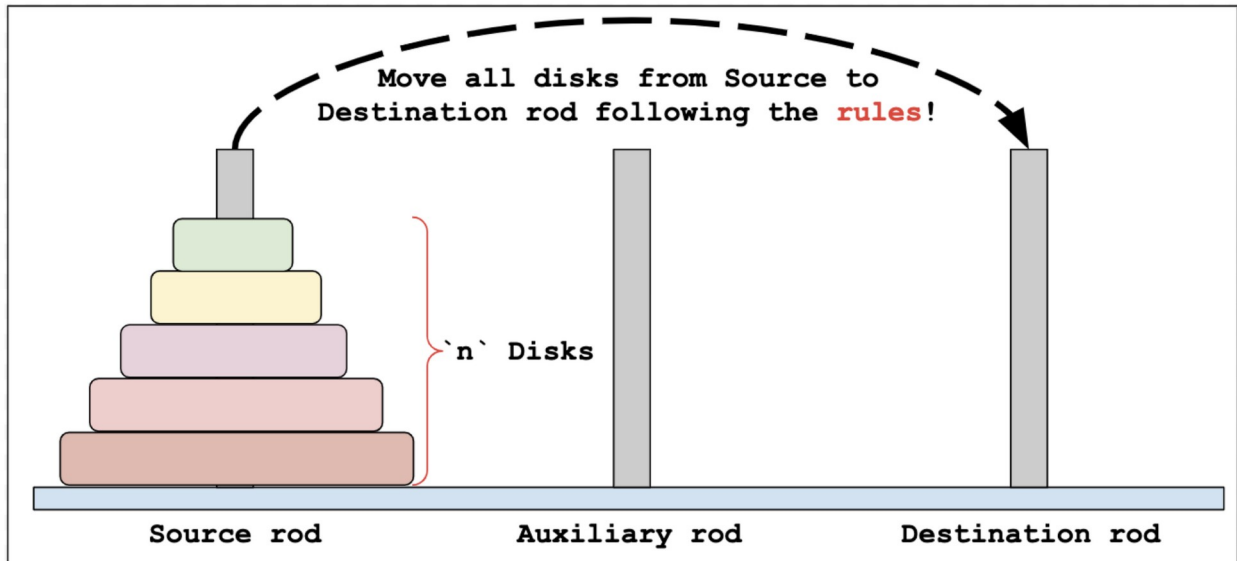
```

"At each step, I save the next node before reversing the pointer, so I don't lose track of the remaining list."



Detecting Loops in Linked Lists In this notebook, you'll implement a function that detects if a loop exists in a linked list. The way we'll do this is by having two pointers, called "runners", moving through the list at different rates. Typically we have a "slow" runner which moves at one node per step and a "fast" runner that moves at two nodes per step.

If a loop exists in the list, the fast runner will eventually move behind the slow runner as it moves to the beginning of the loop. Eventually it will catch up to the slow runner and both runners will be pointing to the same node at the same time. If this happens then you know there is a loop in the linked list. Below is an example where we have a slow runner (the green arrow) and a fast runner (the red arrow).



```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class LinkedList:
    def __init__(self, init_list=None):
        self.head = None
        if init_list:
            for value in init_list:
                self.append(value)

    def append(self, value):
        if self.head is None:
            self.head = Node(value)
            return

        # Move to the tail (the last node)
        node = self.head
        while node.next:
            node = node.next

        node.next = Node(value)
        return

list_with_loop = LinkedList([2, -1, 3, 0, 5])

# Creating a loop where the last node points back to the second node
loop_start = list_with_loop.head.next

node = list_with_loop.head
while node.next:
```

```

    node = node.next
node.next = loop_start
# Solution
def iscircular(linked_list):
    if linked_list.head is None:
        return False

    slow = linked_list.head
    fast = linked_list.head

    while fast and fast.next:
        # slow pointer moves one node
        slow = slow.next
        # fast pointer moves two nodes
        fast = fast.next.next

        if slow == fast:
            return True

    # If we get to a node where fast doesn't have a next node or
    # doesn't exist itself,
    # the list has an end and isn't circular
    return False

# Test Cases

# Create another circular linked list
small_loop = LinkedList([0])
small_loop.head.next = small_loop.head

print ("Pass" if iscircular(list_with_loop) else "Fail")
# Pass
print ("Pass" if iscircular(LinkedList([-4, 7, 2, 5, -1])) else
"Fail") # Fail
print ("Pass" if iscircular(LinkedList([1])) else "Fail")
# Fail
print ("Pass" if iscircular(small_loop) else "Fail")
# Pass
print ("Pass" if iscircular(LinkedList([])) else "Fail")
# Fail

Pass
Fail
Fail
Pass
Fail

```

Create a sorted linked list Given a stream of random integers, create a linked list that is always sorted from ascending order (lowest to highest). What's the computational complexity of adding an element in this way?

```
# Use this class as the nodes in your linked list
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

    def __repr__(self):
        return str(self.value)

class SortedLinkedList:
    def __init__(self):
        self.head = None

    def append(self, value):
        if self.head is None:
            self.head = Node(value)
            return

        if value < self.head.value:
            node = Node(value)
            node.next = self.head
            self.head = node
            return

        node = self.head
        while node.next is not None and value >= node.next.value:
            node = node.next

        new_node = Node(value)
        new_node.next = node.next
        node.next = new_node

        return None

# Test cases
linked_list = SortedLinkedList()
linked_list.append(3)
print("Pass" if (linked_list.head.value == 3) else "Fail")

linked_list.append(2)
print("Pass" if (linked_list.head.value == 2) else "Fail")

linked_list.append(4)
node = linked_list.head.next.next
print("Pass" if (node.value == 4) else "Fail")
```

Pass
Pass
Pass

Problem Statement

You are given a non-negative number in the form of list elements. For example, the number 123 would be provided as `arr = [1, 2, 3]`. Add one to the number and return the output in the form of a new list.

Example 1:

- `input = [1, 2, 3]`
- `output = [1, 2, 4]`

Example 2:

- `input = [1, 2, 9]`
- `output = [1, 3, 0]`

Example 3:

- `input = [9, 9, 9]`
- `output = [1, 0, 0, 0]`

Challenge: One way to solve this problem is to convert the input array into a number and then add one to it. For example, if we have `input = [1, 2, 3]`, you could solve this problem by creating the number 123 and then separating the digits of the output number 124.

But can you solve it in some other way?

Exercise - Write your function definition here.

Note - Try proposing a non-recursive solution. We will see a recursive solution in the next lesson "Recursion".

```
def add_one(arr):  
    borrow = 1  
    for i in range(len(arr), 0, -1):  
        digit = borrow + arr[i - 1]  
        borrow = digit // 10  
        if borrow == 0:  
            arr[i - 1] = digit  
            break  
        else:  
            arr[i - 1] = digit % 10  
    arr = [borrow] + arr  
    position = 0  
    while arr[position] == 0:
```

```

        position += 1
    return arr[position:]

# A helper function for Test Cases
def test_function(test_case):
    arr = test_case[0]
    solution = test_case[1]

    output = add_one(arr)
    print("Output:", output)
    for index, element in enumerate(output):
        if element != solution[index]:
            print("Fail")
            return
    print("Pass")

```

"I start with a carry of 1, move backward through the digits adding the carry. If the sum is less than 10, I update the digit and stop. Otherwise, I set the digit to sum mod 10 and carry 1 to the next digit. Finally, if carry remains, I add it to the front and remove any leading zeros."

```

arr = [1, 2, 9]
solution = [1, 3, 0]
test_case = [arr, solution]
test_function(test_case)

Initial array: [1, 2, 9]
Index 2: digit = 10, borrow = 1
Updated arr[2] to 0 (digit % 10)
Index 1: digit = 3, borrow = 0
Updated arr[1] to 3, breaking loop
Array after adding borrow in front: [0, 1, 3, 0]
Returning array starting from position 1: [1, 3, 0]
Output: [1, 3, 0]
Pass

# Test Case 1
arr = [0]
solution = [1]
test_case = [arr, solution]
test_function(test_case)

Output: [1]
Pass

# Test Case 2
arr = [1, 2, 3]
solution = [1, 2, 4]
test_case = [arr, solution]
test_function(test_case)

```

Output: [1, 2, 4]

Pass

Test Case 3

arr = [9, 9, 9]

solution = [1, 0, 0, 0]

test_case = [arr, solution]

test_function(test_case)

Output: [1, 0, 0, 0]

Pass

Problem Statement

You have been given an array of `length = n`. The array contains integers from `0` to `n - 2`. Each number in the array is present exactly once except for one number which is present twice. Find and return this duplicate number present in the array

Example:

- `arr = [0, 2, 3, 1, 4, 5, 3]`
- `output = 3` (because 3 is present twice)

The expected time complexity for this problem is `O(n)` and the expected space-complexity is `O(1)`.

Solution

'''

Notice carefully that

- 1. All the elements of the array are always non-negative*
- 2. If array length = n, then elements would start from 0 to (n-2), i.e. Natural numbers 0,1,2,3,4,5...(n-2)*
- 3. There is only SINGLE element which is present twice.*

Therefore let's find the sum of all elements (current_sum) of the original array, and find the sum of first (n-2) Natural numbers (expected_sum).

Trick:

The second occurrence of a particular number (say 'x') is actually occupying the space

that would have been utilized by the number (n-1). This leads to:

current_sum = 0 + 1 + 2 + 3 + + (n-2) + x

expected_sum = 0 + 1 + 2 + 3 + + (n-2)

current_sum - expected_sum = x

Tada!!! :)

'''

def duplicate_number(arr):

 current_sum = 0

```

expected_sum = 0

# Traverse the original array in the forward direction
for num in arr:
    current_sum += num

# Traverse from 0 to (length of array-1) to get the expected_sum
# Alternatively, you can use the formula for sum of an Arithmetic
Progression to get the expected_sum

# The argument of range() functions are:
# starting index [OPTIONAL], ending index (non exclusive), and the
increment/decrement size [OPTIONAL]
# It means that if the array length = n, loop will run from 0 to
(n-2)
for i in range(len(arr) - 1):
    expected_sum += i

# The difference between the
return current_sum - expected_sum

def test_function(test_case):
    arr = test_case[0]
    solution = test_case[1]
    output = duplicate_number(arr)
    if output == solution:
        print("Pass")
    else:
        print("Fail")

arr = [0, 0]
solution = 0

test_case = [arr, solution]
test_function(test_case)

Pass

arr = [0, 2, 3, 1, 4, 5, 3]
solution = 3

test_case = [arr, solution]
test_function(test_case)

Pass

arr = [0, 1, 5, 4, 3, 2, 0]
solution = 0

```



```
test_case = [arr, solution]
test_function(test_case)
```

Pass

```
arr = [0, 1, 5, 5, 3, 2, 4]
solution = 5
```

```
test_case = [arr, solution]
test_function(test_case)
```

Pass

Problem Statement

You have been given an array containing numbers. Find and return the largest sum in a contiguous subarray within the input array.

Example 1:

- `arr = [1, 2, 3, -4, 6]`
- The largest sum is `8`, which is the sum of all elements of the array.

Example 2:

- `arr = [1, 2, -5, -4, 1, 6]`
- The largest sum is `7`, which is the sum of the last two elements of the array.

```
# Solution
'''
```

The Idea:

1. We have to find the sum of "contiguous" subarray, therefore we must not change the order of array elements.

2. Let `'current_sum'` denotes the sum of a subarray, and `'max_sum'` denotes the maximum value of `'current_sum'`.

3. LOOP STARTS: For each element of the array, update the `'current_sum'` with the MAXIMUM of either:

- The element added to the `'current_sum'` (denotes the addition of the element to the current subarray)

- The element itself (denotes the starting of a new subarray)

- Update (overwrite) `'max_sum'`, if it is lower than the updated `'current_sum'`

4. Return `'max_sum'`

```
'''
```

```
def max_sum_subarray(arr):
```

```
    current_sum = arr[0] # 'current_sum' denotes the sum of a subarray
    max_sum = arr[0]     # 'max_sum' denotes the maximum value of
    'current_sum' ever
```

```

# Loop from VALUE at index position 1 till the end of the array
for element in arr[1:]:
    '''
    # Compare (current_sum + element) vs (element)
    # If (current_sum + element) is higher, it denotes the
    addition of the element to the current subarray
    # If (element) alone is higher, it denotes the starting of a
    new subarray
    '''
    current_sum = max(current_sum + element, element)

    # Update (overwrite) `max_sum`, if it is lower than the
    updated `current_sum`
    max_sum = max(current_sum, max_sum)

return max_sum

def test_function(test_case):
    arr = test_case[0]
    solution = test_case[1]

    output = max_sum_subarray(arr)
    if output == solution:
        print("Pass")
    else:
        print("Fail")

arr= [1, 2, 3, -4, 6]
solution= 8 # sum of array

test_case = [arr, solution]
test_function(test_case)

Pass

arr = [1, 2, -5, -4, 1, 6]
solution = 7 # sum of last two elements

test_case = [arr, solution]
test_function(test_case)

Pass

arr = [-12, 15, -13, 14, -1, 2, 1, -5, 4]
solution = 18 # sum of subarray = [15, -13, 14, -1, 2, 1]

test_case = [arr, solution]
test_function(test_case)

Pass

```

Problem Statement

Find and return the `nth` row of Pascal's triangle in the form a list. `n` is 0-based.

For example, if `n = 4`, then `output = [1, 4, 6, 4, 1]`.

To know more about Pascal's triangle: <https://www.mathsisfun.com/pascals-triangle.html>

Solution

'''

Points to note:

- 1. We have to return a list.*
 - 2. The elements of n^{th} row are made up of elements of $(n-1)^{\text{th}}$ row. This comes up till the 1^{st} row. We will follow a top-down approach.*
 - 3. Except for the first and last element, any other element at position `j` in the current row is the sum of elements at position `j` and `j-1` in the previous row.*
 - 4. Be careful about the edge cases, example, an index should never be a NEGATIVE at any point of time.*
- '''*

`def nth_row_pascal(n):`

`if n == 0:`
 `return [1]`

`current_row = [1] # First row`

`''' Loop from 1 to n; `i` denotes the row number'''`
 `for i in range(1, n + 1):`
 `# Set the `current_row` from previous iteration as the`
 ``previous_row``
 `previous_row = current_row`

 `# Let's build the fresh current_row gradually`
 `current_row = [1] # add the default first element at the 0th`
 `index of the `ith` row`

 `'''Loop from 1 to (i-1); `j` denotes the index of an element`
 `with in the `ith` row'''`
 `# Example, for 5th row we have considered n=4,`
 `# we will iterate index from 1 to 3, because`
 `# the default element at the 0th index has already been added`
 `for j in range(1, i):`

 `# An element at position `j` in the current row is the`
 `# sum of elements at position `j` and `j-1` in the`
 `previous row.`

```

        print(f"Calculating element at position {j} for row {i}")
        next_number = previous_row[j] + previous_row[j - 1]

        # Append the new element to the current_row
        current_row.append(next_number)

    current_row.append(1) # append the default last element
    return current_row

```

Every row builds on the previous row, and:

The first and last numbers in every row are always 1.

Every middle number is the sum of the two numbers above it (from the previous row): $\text{current}[j] = \text{previous}[j-1] + \text{previous}[j]$

```

def test_function(test_case):
    n = test_case[0]
    solution = test_case[1]
    output = nth_row_pascal(n)
    if solution == output:
        print("Pass")
    else:
        print("Fail")

```

```

n = 1
solution = [1, 1]

```

```

test_case = [n, solution]
test_function(test_case)

```

Pass

```

n = 2
solution = [1, 2, 1]

```

```

test_case = [n, solution]
test_function(test_case)

```

Calculating element at position 1 for row 2
Pass

```

n = 3
solution = [1, 3, 3, 1]

```

```

test_case = [n, solution]
test_function(test_case)

```

Calculating element at position 1 for row 2
Calculating element at position 1 for row 3

```
Calculating element at position 2 for row 3
Pass
```

```
n = 4
solution = [1, 4, 6, 4, 1]
```

```
test_case = [n, solution]
test_function(test_case)
```

```
Calculating element at position 1 for row 2
Calculating element at position 1 for row 3
Calculating element at position 2 for row 3
Calculating element at position 1 for row 4
Calculating element at position 2 for row 4
Calculating element at position 3 for row 4
Pass
```

Problem Statement

You are given the head of a linked list and two integers, *i* and *j*. You have to retain the first *i* nodes and then delete the next *j* nodes. Continue doing so until the end of the linked list.

Example:

- linked-list = 1 2 3 4 5 6 7 8 9 10 11 12
- *i* = 2
- *j* = 3
- Output = 1 2 6 7 11 12

```
# LinkedList Node class for your reference
```

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
# Solution
```

```
"""
:param: head - head of linked list
:param: i - first `i` nodes that are to be skipped
:param: j - next `j` nodes that are to be deleted
return - return the updated head of the linked list
"""
...
```

The Idea:

Traverse the Linkedlist. Make use of two references - `current` and `previous`.

- Skip `*i*-1` nodes. Keep incrementing the `current`. Mark the `*i*-1`th node as `previous`.
- Delete next `*j*` nodes. Keep incrementing the `current`.
- Connect the `previous.next` to the `current`

```

'''
def skip_i_delete_j(head, i, j):
    # Edge case - Skip 0 nodes (means Delete all nodes)
    if i == 0:
        return None

    # Edge case - Delete 0 nodes
    if j == 0:
        return head

    # Invalid input
    if head is None or j < 0 or i < 0:
        return head

    # Helper references
    current = head
    previous = None

    # Traverse - Loop untill there are Nodes available in the
    # LinkedList
    while current:

        '''skip (i - 1) nodes'''
        for _ in range(i - 1):
            if current is None:
                return head
            current = current.next
        previous = current
        current = current.next

        '''delete next j nodes'''
        for _ in range(j):
            if current is None:
                break
            next_node = current.next
            current = next_node

        '''Connect the `previous.next` to the `current`'''
        previous.next = current

    # Loop ends

    return head

# helper functions for testing purpose
def create_linked_list(arr):
    if len(arr)==0:
        return None
    head = Node(arr[0])
    tail = head

```

```

    for data in arr[1:]:
        tail.next = Node(data)
        tail = tail.next
    return head

def print_linked_list(head):
    while head:
        print(head.data, end=' ')
        head = head.next
    print()

def test_function(test_case):
    head = test_case[0]
    i = test_case[1]
    j = test_case[2]
    solution = test_case[3]

    temp = skip_i_delete_j(head, i, j)
    index = 0
    try:
        while temp is not None:
            if temp.data != solution[index]:
                print("Fail")
                return
            index += 1
            temp = temp.next
        print("Pass")
    except Exception as e:
        print("Fail")

arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
i = 2
j = 2
head = create_linked_list(arr)
solution = [1, 2, 5, 6, 9, 10]
test_case = [head, i, j, solution]
test_function(test_case)

```

Pass

Balanced Parentheses Exercise

In this exercise you are going to apply what you learned about stacks with a real world problem. We will be using stacks to make sure the parentheses are balanced in mathematical expressions such as: $((3^2+8)*(5/2))/(2+6)$. In real life you can see this extend to many things such as text editor plugins and interactive development environments for all sorts of bracket completion checks.

Take a string as an input and return `True` if it's parentheses are balanced or `False` if it is not.

Try to code up a solution and pass the test cases.

```
def equation_checker(equation):
    stack = []
    for char in equation:
        if char == '(':
            stack.append(char)
        elif char == ')':
            if not stack:
                return False # Found a ')' without matching '('
            stack.pop()
    return len(stack) == 0 # True if no unmatched '(' remain

print ("Pass" if (equation_checker('((3^2 + 8)*(5/2))/(2+6)')) else
"Fail")
print ("Pass" if not (equation_checker('((3^2 + 8)*(5/2))/(2+6)'))
else "Fail")
```

Pass
Pass

Reverse Polish Notation

Reverse Polish notation, also referred to as **Polish postfix notation** is a way of laying out operators and operands.

When making mathematical expressions, we typically put arithmetic operators (like $+$, $-$, $*$, and $/$) *between* operands. For example: $5 + 7 - 3 * 8$

However, in Reverse Polish Notation, the operators come *after* the operands. For example: $3 \ 1 \ + \ 4 \ *$

The above expression would be evaluated as $(3 + 1) * 4 = 16$

The goal of this exercise is to create a function that does the following:

- Given a *postfix* expression as input, evaluate and return the correct final answer.

Note: In Python 3, the division operator $/$ is used to perform float division. So for this problem, you should use `int()` after every division to convert the answer to an integer.

```
def evaluate_post_fix(input_list):
    """
    Evaluate the postfix expression to find the answer

    Args:
        input_list (list): List containing the postfix expression
    Returns:
        int: Result of the postfix expression
    """
    stack = []
```



```

for token in input_list:
    if token in ['+', '-', '*', '/']:
        # Pop the top two operands (note the order!)
        b = stack.pop()
        a = stack.pop()

        if token == '+':
            result = a + b
        elif token == '-':
            result = a - b
        elif token == '*':
            result = a * b
        elif token == '/':
            result = int(a / b) # Use int() to truncate towards
zero

        # Push result back to stack
        stack.append(result)
    else:
        # Operand, just push to stack
        stack.append(int(token))

return stack[0]

def test_function(test_case):
    output = evaluate_post_fix(test_case[0])
    print(output)
    if output == test_case[1]:
        print("Pass")
    else:
        print("Fail")

test_case_1 = [["3", "1", "+", "4", "*"], 16]
test_function(test_case_1)

16
Pass

test_case_2 = [["4", "13", "5", "/", "+"], 6]
test_function(test_case_2)

6
Pass

test_case_3 = [["10", "6", "9", "3", "+", "-11", "*", "/", "*", "17",
"+", "5", "+"], 22]
test_function(test_case_3)

```

22
Pass

Problem Statement

Reverse a stack. If your stack initially has 1, 2, 3, 4 (4 at the top and 1 at the bottom), after reversing the order must be 4, 3, 2, 1 (4 at the bottom and 1 at the top).

```
def insert_at_bottom(stack, item, depth=0):
    indent = "  " * depth # indentation for better readability in
nested calls

    print(f"{indent}Called insert_at_bottom(stack={stack},
item={item})")

    if not stack:
        stack.append(item)
        print(f"{indent}Stack was empty, appended item {item} ->
stack: {stack}")
    else:
        temp = stack.pop()
        print(f"{indent}Popped top item {temp} -> stack now: {stack}")

        insert_at_bottom(stack, item, depth + 1)

        stack.append(temp)
        print(f"{indent}Pushed back item {temp} -> stack now:
{stack}")

def reverse_stack(stack):
    if stack:
        temp = stack.pop()
        reverse_stack(stack)
        insert_at_bottom(stack, temp)

test_case_1 = [1, 2, 3, 4]
reverse_stack(test_case_1)
print("test_case_1:", test_case_1)

Called insert_at_bottom(stack=[], item=1)
Stack was empty, appended item 1 -> stack: [1]
Called insert_at_bottom(stack=[1], item=2)
Popped top item 1 -> stack now: []
    Called insert_at_bottom(stack=[], item=2)
    Stack was empty, appended item 2 -> stack: [2]
    Pushed back item 1 -> stack now: [2, 1]
    Called insert_at_bottom(stack=[2, 1], item=3)
    Popped top item 1 -> stack now: [2]
```

```

Called insert_at_bottom(stack=[2], item=3)
Popped top item 2 -> stack now: []
Called insert_at_bottom(stack=[], item=3)
Stack was empty, appended item 3 -> stack: [3]
Pushed back item 2 -> stack now: [3, 2]
Pushed back item 1 -> stack now: [3, 2, 1]
Called insert_at_bottom(stack=[3, 2, 1], item=4)
Popped top item 1 -> stack now: [3, 2]
Called insert_at_bottom(stack=[3, 2], item=4)
Popped top item 2 -> stack now: [3]
Called insert_at_bottom(stack=[3], item=4)
Popped top item 3 -> stack now: []
Called insert_at_bottom(stack=[], item=4)
Stack was empty, appended item 4 -> stack: [4]
Pushed back item 3 -> stack now: [4, 3]
Pushed back item 2 -> stack now: [4, 3, 2]
Pushed back item 1 -> stack now: [4, 3, 2, 1]
test_case_1: [4, 3, 2, 1]

```

```

class LinkedListNode:

    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:

    def __init__(self):
        self.num_elements = 0
        self.head = None

    def push(self, data):
        new_node = LinkedListNode(data)
        if self.head is None:
            self.head = new_node
        else:
            new_node.next = self.head
            self.head = new_node
        self.num_elements += 1

    def pop(self):
        if self.is_empty():
            return None
        temp = self.head.data
        self.head = self.head.next
        self.num_elements -= 1
        return temp

    def top(self):
        if self.head is None:

```

```

        return None
    return self.head.data

def size(self):
    return self.num_elements

def is_empty(self):
    return self.num_elements == 0
def __str__(self):
    values = []
    current = self.head
    while current:
        values.append(str(current.data))
        current = current.next
    return "Stack(bottom -> top): " + " -> " + ".join(reversed(values))

def reverse_stack(stack):
    holder_stack = Stack()
    while not stack.is_empty():
        popped_element = stack.pop()
        holder_stack.push(popped_element)
    _reverse_stack_recursion(stack, holder_stack)

def _reverse_stack_recursion(stack, holder_stack):
    if holder_stack.is_empty():
        return
    print("Holder Stack:", holder_stack)
    popped_element = holder_stack.pop()
    _reverse_stack_recursion(stack, holder_stack)
    stack.push(popped_element)
    print("Stack after pushing:", stack, "Size:", stack.size())

def test_function(test_case):
    stack = Stack()
    for num in test_case:
        stack.push(num)

    reverse_stack(stack)
    index = 0
    while not stack.is_empty():
        popped = stack.pop()
        if popped != test_case[index]:
            print("Fail")
            return
        else:
            index += 1
    print("Pass")

```

```

test_case_1 = [1, 2, 3, 4]
test_function(test_case_1)

Holder Stack: Stack(bottom -> top): 4 -> 3 -> 2 -> 1
Holder Stack: Stack(bottom -> top): 4 -> 3 -> 2
Holder Stack: Stack(bottom -> top): 4 -> 3
Holder Stack: Stack(bottom -> top): 4
Stack after pushing: Stack(bottom -> top): 4 Size: 1
Stack after pushing: Stack(bottom -> top): 4 -> 3 Size: 2
Stack after pushing: Stack(bottom -> top): 4 -> 3 -> 2 Size: 3
Stack after pushing: Stack(bottom -> top): 4 -> 3 -> 2 -> 1 Size: 4
Pass

```

Problem Statement

Given an input string consisting of only { and }, figure out the minimum number of reversals required to make the brackets balanced.

For example:

- For input_string = "}}}}", the number of reversals required is 2.
- For input_string = "{}{}", the number of reversals required is 1.

If the brackets cannot be balanced, return -1 to indicate that it is not possible to balance them.

```

def minimum_bracket_reversals(input_string):
    # If length is odd, can't be balanced
    if len(input_string) % 2 != 0:
        return -1

    stack = []

    for char in input_string:
        if char == '{':
            stack.append(char)
        else: # char == '}'
            if stack and stack[-1] == '{':
                stack.pop()
            else:
                stack.append(char)

    # Now stack contains only unbalanced brackets
    # Count the number of '{' and '}' in stack
    open_braces = stack.count('{')
    close_braces = len(stack) - open_braces

```

```
    # Each two of the same kind need 1 reversal, so (x + 1) // 2  
    handles both even and odd counts
```

```
    return (open_braces + 1) // 2 + (close_braces + 1) // 2
```

```
print(minimum_bracket_reversals("}}}"))    # Output: 2  
print(minimum_bracket_reversals("{}{}"))  # Output: 2  
print(minimum_bracket_reversals("{}{}{}")) # Output: 2  
print(minimum_bracket_reversals("{}{}{}{}")) # Output: 1  
print(minimum_bracket_reversals("{}{}{}{}{}")) # Output: -1
```

```
2  
2  
2  
1  
-1
```

```
def minimum_bracket_reversals(input_string):  
    if len(input_string) % 2 == 1:  
        return -1
```

```
    stack = []  
    count = 0
```

```
    for bracket in input_string:  
        if not stack:  
            stack.append(bracket)  
        else:  
            if stack[-1] == '{' and bracket == '}':  
                stack.pop() # balanced pair, remove  
            else:  
                stack.append(bracket)
```

```
    # Now stack has only unbalanced brackets  
    # Process two at a time
```

```
    while stack:  
        first = stack.pop()  
        second = stack.pop()  
        if first == '}' and second == '{':  
            count += 1  
        elif first == '{' and second == '{':  
            count += 1  
        else: # one '{' and one '}'  
            count += 2
```

```
    return count
```

```
print(minimum_bracket_reversals("}}}"))    # Output: 2  
print(minimum_bracket_reversals("{}{}{}")) # Output: 2  
print(minimum_bracket_reversals("{}{}{}{}")) # Output: 2  
print(minimum_bracket_reversals("{}{}{}{}{}")) # Output: 1  
print(minimum_bracket_reversals("{}{}{}{}{}{}")) # Output: -1
```

```
2
2
2
1
-1
```

Reversed Queue

Write a function that takes a queue as an input and returns a reversed version of it.

```
from collections import deque

def reverse_queue(queue, depth=0):
    indent = "  " * depth
    if not queue:
        print(f"{indent}Queue is empty, returning.")
        return queue

    item = queue.popleft()
    print(f"{indent}Dequeued item: {item}, current queue: {list(queue)}")

    reverse_queue(queue, depth + 1)

    queue.append(item)
    print(f"{indent}Enqueued item: {item}, queue now: {list(queue)}")

    return queue

q = deque([1, 2, 3, 4, 5])
reversed_q = reverse_queue(q)
print(reversed_q) # Output: deque([5, 4, 3, 2, 1])

Dequeued item: 1, current queue: [2, 3, 4, 5]
Dequeued item: 2, current queue: [3, 4, 5]
Dequeued item: 3, current queue: [4, 5]
Dequeued item: 4, current queue: [5]
Dequeued item: 5, current queue: []
Queue is empty, returning.
Enqueued item: 5, queue now: [5]
Enqueued item: 4, queue now: [5, 4]
Enqueued item: 3, queue now: [5, 4, 3]
Enqueued item: 2, queue now: [5, 4, 3, 2]
Enqueued item: 1, queue now: [5, 4, 3, 2, 1]
deque([5, 4, 3, 2, 1])

def sum_array(array):
    # Base Case
```

```

    if len(array) == 1:
        return array[0]

    return array[0] + sum_array(array[1:])

arr = [1, 2, 3, 4]
print(sum_array(arr))

10

import matplotlib.pyplot as plt
import statistics
import time
%matplotlib inline

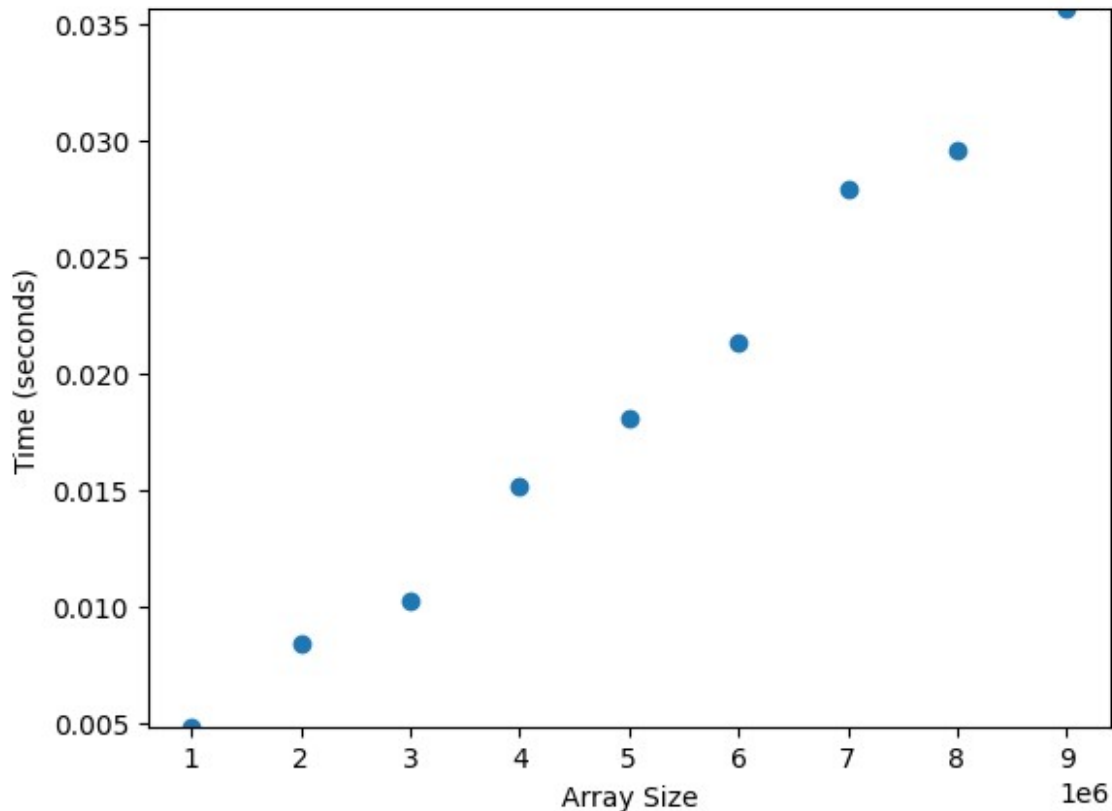
n_steps = 10
step_size = 1000000
array_sizes = list(range(step_size, n_steps*step_size, step_size))
big_array = list(range(n_steps*step_size))
times = []

# Calculate the time it takes for the slice function to run with
different sizes of k
for array_size in array_sizes:
    start_time = time.time()
    big_array[:array_size]
    times.append(time.time() - start_time)

# Graph the results
plt.scatter(x=array_sizes, y=times)
plt.ylim(top=max(times), bottom=min(times))
plt.xlabel('Array Size')
plt.ylabel('Time (seconds)')
plt.plot()

[]

```

```
def sum_array_index(array, index):
    # Base Cases
    if len(array) - 1 == index:
        return array[index]

    return array[index] + sum_array_index(array, index + 1)

arr = [1, 2, 3, 4]
print(sum_array_index(arr, 0))

10

import matplotlib.pyplot as plt
import statistics
import time

n_steps = 10
step_size = 200
array_sizes = list(range(step_size, n_steps*step_size, step_size))
big_array = list(range(n_steps*step_size))
sum_array_times = []
sum_array_index_times = []

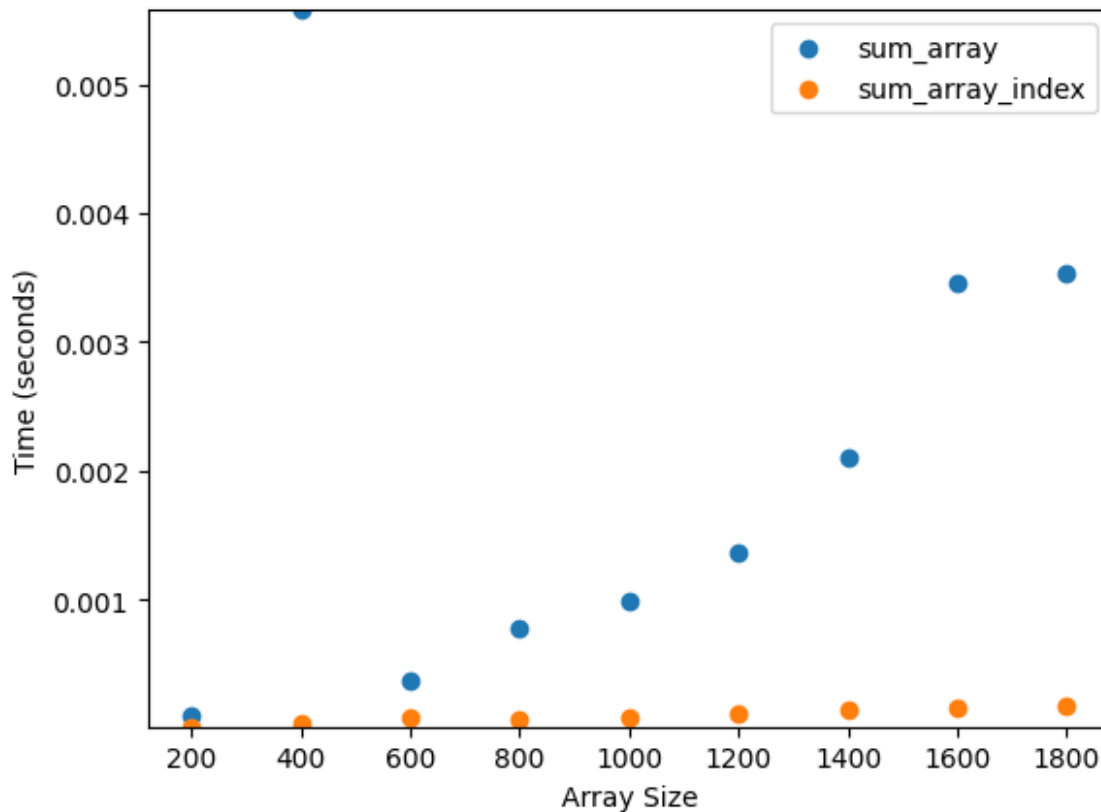
for array_size in array_sizes:
    subset_array = big_array[:array_size]
```

```
start_time = time.time()
sum_array(subset_array)
sum_array_times.append(time.time() - start_time)

start_time = time.time()
sum_array_index(subset_array, 0)
sum_array_index_times.append(time.time() - start_time)

plt.scatter(x=array_sizes, y=sum_array_times, label='sum_array')
plt.scatter(x=array_sizes, y=sum_array_index_times,
label='sum_array_index')
plt.ylim(
    top=max(sum_array_times + sum_array_index_times),
    bottom=min(sum_array_times + sum_array_index_times))
plt.legend()
plt.xlabel('Array Size')
plt.ylabel('Time (seconds)')
plt.plot()

[]
```



The **factorial** function is a mathematical function that multiplies a given number, n , and all of the whole numbers from n down to 1.

For example, if n is 4 then we will get:

$$4 * 3 * 2 * 1 = 24$$

This is often notated using an exclamation point, as in $4!$ (which would be read as "four factorial").

$$\text{So } 4! = 4 * 3 * 2 * 1 = 24$$

More generally, we can say that for any input n :

$$n! = n * (n - 1) * (n - 2) \dots 1$$

If you look at this more closely, you will find that the factorial of any number is the product of that number and the factorial of the next smallest number. In other words:

$$n! = n * (n - 1)!$$

Notice that this is recursive, meaning that we can solve for the factorial of any given number by first solving for the factorial of the next smallest number, and the next smallest number, and the next smallest number, and so on, until we reach 1.

If you were to write a Python function called `factorial` to calculate the factorial of a number, then we could restate what we said above as follows:

```
factorial(n) = n * factorial(n-1)
```

So that is the goal of this exercise: To use recursion to write a function that will take a number and return the factorial of that number.

For example, you should be able to call your function with `factorial(4)` and get back 24.

Note: By definition, $0! = 1$

```
# Solution

def factorial(n):
    """
    Calculate n!

    Args:
        n(int): factorial to be computed
    Returns:
        n!
    """

    if n == 0:
        return 1 # by definition of 0!
    return n * factorial(n-1)
```

```
print ("Pass" if (1 == factorial(0)) else "Fail")
print ("Pass" if (1 == factorial(1)) else "Fail")
print ("Pass" if (120 == factorial(5)) else "Fail")
```

Pass
Pass
Pass

Reversing a String

The goal in this notebook will be to get practice with a problem that is frequently solved by recursion: Reversing a string.

Note that Python has a built-in function that you could use for this, but the goal here is to avoid that and understand how it can be done using recursion instead.

```
def reverse_string(input):
    # Base condition
    if len(input) == 0:
        return ""
    else:
        first_char = input[0]
        sub_string = input[1:] # directly slice the string
        reversed_substring = reverse_string(sub_string)
        return reversed_substring + first_char

# Test Cases

print ("Pass" if (" " == reverse_string("")) else "Fail")
print ("Pass" if ("cba" == reverse_string("abc")) else "Fail")
```

Pass
Pass

A **palindrome** is a word that is the reverse of itself—that is, it is the same word when read forwards and backwards.

For example:

- "madam" is a palindrome
- "abba" is a palindrome
- "cat" is not
- "a" is a trivial case of a palindrome

The goal of this exercise is to use recursion to write a function `is_palindrome` that takes a string as input and checks whether that string is a palindrome. (Note that this problem can also be solved with a non-recursive solution, but that's not the point of this exercise.)

```

# Solution

def is_palindrome(input):
    """
    Return True if input is palindrome, False otherwise.

    Args:
        input(str): input to be checked if it is palindrome
    """

    # Termination / Base condition
    if len(input) <= 1:
        return True
    else:
        first_char = input[0]
        last_char = input[-1]

        # sub_input is input with first and last char removed
        sub_input = input[1:-1]

        # recursive call, if first and last char are identical, else
    return False
        return (first_char == last_char) and is_palindrome(sub_input)

print ("Pass" if (is_palindrome("")) else "Fail")
print ("Pass" if (is_palindrome("a")) else "Fail")
print ("Pass" if (is_palindrome("madam")) else "Fail")
print ("Pass" if (is_palindrome("abba")) else "Fail")
print ("Pass" if not (is_palindrome("Udacity")) else "Fail")

Pass
Pass
Pass
Pass
Pass

```

Let us solve a problem that we solved previously in the last lesson without using recursion. It will make you realize that recursion can make our code look simpler.

Problem Statement

You are given a non-negative number in the form of list elements. For example, the number 123 would be provided as `arr = [1, 2, 3]`. Add one to the number and return the output in the form of a new list.

Example 1:

- `input = [1, 2, 3]`
- `output = [1, 2, 4]`

Example 2:

- `input = [1, 2, 9]`
- `output = [1, 3, 0]`

Example 3:

- `input = [9, 9, 9]`
- `output = [1, 0, 0, 0]`

Exercise - Write the RECURSIVE function definition here

```
# Recursive Solution
def add_one(arr):
    """
    :param: arr - list of digits representing some number x
    return a list with digits representing (x + 1)
    """
    # Base case
    if arr == [9]:
        return [1, 0]

    # A simple case, where we just need to increment the last digit
    if arr[-1] < 9:
        arr[-1] += 1
    # Case when the last digit is 9.
    else:
        '''Recursive call'''
        # We have used arr[:-1], that means all elements of the list
        except the last one.
        # Example, for original input arr=[1,2,9], we will pass [1,2]
        in next call.
        arr = add_one(arr[:-1]) + [0]
        print("After recursive call, arr:", arr)

    return arr

# A helper function for Test Cases
def test_function(test_case):
    arr = test_case[0]
    solution = test_case[1]

    output = add_one(arr)
    for index, element in enumerate(output):
        if element != solution[index]:
            print("Fail")
            return
    print("Pass")

# Test Case 1
arr = [0]
solution = [1]
```

```
test_case = [arr, solution]
test_function(test_case)
```

Pass

Test Case 2

```
arr = [1, 2, 3]
solution = [1, 2, 4]
test_case = [arr, solution]
test_function(test_case)
```

Pass

Test Case 3

```
arr = [9, 9, 9]
solution = [1, 0, 0, 0]
test_case = [arr, solution]
test_function(test_case)
```

After recursive call, arr: [1, 0, 0]

After recursive call, arr: [1, 0, 0, 0]

Pass

Test Case 3

```
arr = [7, 9, 9, 9]
solution = [8, 0, 0, 0]
test_case = [arr, solution]
test_function(test_case)
```

After recursive call, arr: [8, 0]

After recursive call, arr: [8, 0, 0]

After recursive call, arr: [8, 0, 0, 0]

Pass

Permutation

Question - Let's use recursion to help us solve the following permutation problem:

Given a list of items, the goal is to find all of the permutations of that list. For example, Given a list like: [0, 1, 2] Permutations: [[0, 1, 2], [0, 2, 1], [1, 0, 2], [1, 2, 0], [2, 0, 1], [2, 1, 0]] Notice that the expected output is a list of permutation with each permuted item being represented by a list. Such an object that contains other object is called "compound" object.

The Idea Build a compoundList incrementally starting with a blank list, and permute (add) each element of original input list at all possible positions.

For example, take [0, 1, 2] as the original input list:

1. Start with a blank compoundList `[[[]]`. This is actually the last call of recursive function stack. Pick the element 2 of original input list, making the compoundList as `[[2]]`
2. Pick next element 1 of original input list, and add this element at position 0, and 1 for each list of previous compoundList. **We will require to create copy of all lists of previous compoundList, and add the new element.** Now, the compoundList will become `[[1, 2], [2, 1]]`.
3. Pick next element 0 of original input list, and add this element at position 0, 1, and 2 for each list of previous compoundList. Now, the compoundList will become `[[0, 1, 2], [1, 0, 2], [1, 2, 0], [0, 2, 1], [2, 0, 1], [2, 1, 0]]`.

```
import copy

def permute(inputList):
    finalCompoundList = []

    if len(inputList) == 0:
        finalCompoundList.append([])
    else:
        first_element = inputList[0]
        rest_list = inputList[1:]
        sub_compoundList = permute(rest_list)

        for aList in sub_compoundList:
            for j in range(0, len(aList) + 1):
                bList = copy.deepcopy(aList)
                bList.insert(j, first_element)
                finalCompoundList.append(bList)

    return finalCompoundList

import copy

def permute(inputList, depth=0):
    indent = "  " * depth
    print(f"{indent}permute called with inputList={inputList}")

    finalCompoundList = []

    if len(inputList) == 0:
        print(f"{indent}Base case reached: inputList is empty")
        finalCompoundList.append([])
    else:
        first_element = inputList[0]
        rest_list = inputList[1:]
        print(f"{indent}first_element: {first_element}")
        print(f"{indent}rest_list: {rest_list}")
```



```

        sub_compoundList = permute(rest_list, depth + 1)
        print(f"{indent}sub_compoundList (permutations of
{rest_list}): {sub_compoundList}")

        for aList in sub_compoundList:
            for j in range(0, len(aList) + 1):
                bList = copy.deepcopy(aList)
                bList.insert(j, first_element)
                print(f"{indent}Inserting {first_element} at position
{j} in {aList} -> {bList}")
                finalCompoundList.append(bList)

        print(f"{indent}Returning finalCompoundList: {finalCompoundList}")
        return finalCompoundList

# Test Cases

# Helper Function
def check_output(output, expected_output):
    """
    Return True if output and expected_output
    contains the same lists, False otherwise.

    Note that the ordering of the list is not important.

    Examples:
        check_output([ [0, 1], [1, 0] ], [ [1, 0], [0, 1] ]) returns
True

    Args:
        output(list): list of list
        expected_output(list): list of list

    Returns:
        bool
    """
    o = copy.deepcopy(output) # so that we don't mutate input
    e = copy.deepcopy(expected_output) # so that we don't mutate
input

    o.sort()
    e.sort()
    return o == e

# print ("Pass" if (check_output(permute([]), [[]])) else "Fail")
# print ("Pass" if (check_output(permute([0]), [[0]])) else "Fail")
# print ("Pass" if (check_output(permute([0, 1]), [[0, 1], [1, 0]]))
else "Fail")
print ("Pass" if (check_output(permute([0, 1, 2]), [[0, 1, 2], [0, 2,
1], [1, 0, 2], [1, 2, 0], [2, 0, 1], [2, 1, 0]])) else "Fail")

```

```

permute called with inputList=[0, 1, 2]
first_element: 0
rest_list: [1, 2]
  permute called with inputList=[1, 2]
  first_element: 1
  rest_list: [2]
    permute called with inputList=[2]
    first_element: 2
    rest_list: []
      permute called with inputList=[]
      Base case reached: inputList is empty
      Returning finalCompoundList: [[]]
    sub_compoundList (permutations of []): [[]]
    Inserting 2 at position 0 in [] -> [2]
    Returning finalCompoundList: [[2]]
  sub_compoundList (permutations of [2]): [[2]]
  Inserting 1 at position 0 in [2] -> [1, 2]
  Inserting 1 at position 1 in [2] -> [2, 1]
  Returning finalCompoundList: [[1, 2], [2, 1]]
sub_compoundList (permutations of [1, 2]): [[1, 2], [2, 1]]
Inserting 0 at position 0 in [1, 2] -> [0, 1, 2]
Inserting 0 at position 1 in [1, 2] -> [1, 0, 2]
Inserting 0 at position 2 in [1, 2] -> [1, 2, 0]
Inserting 0 at position 0 in [2, 1] -> [0, 2, 1]
Inserting 0 at position 1 in [2, 1] -> [2, 0, 1]
Inserting 0 at position 2 in [2, 1] -> [2, 1, 0]
Returning finalCompoundList: [[0, 1, 2], [1, 0, 2], [1, 2, 0], [0, 2, 1], [2, 0, 1], [2, 1, 0]]
Pass

```

Problem Statement

Given an input string, return all permutations of the string in an array.

Example 1:

- string = 'ab'
- output = ['ab', 'ba']

Example 2:

- string = 'abc'
- output = ['abc', 'bac', 'bca', 'acb', 'cab', 'cba']

Note - Strings are Immutable

Strings in Python are immutable, which means that we cannot overwrite the characters of the String objects. For example:

```
str1 = "Hello"
str1[0] = 'K' # Try changing the first
character
```

will lead to

```
TypeError: 'str' object does not support item assignment
```

We can only re-assign the variable to a new value (string), as follows:

```
str1 = "Udacity" # re-assignment
str2 = "Welcome to the " + str1[3:] # Returns 'Welcome to the city'
```

Therefore, we do not require a deep copy in this exercise, as it was the case in our last example of list permutation.

The Idea Starting with a blank list, add each character of original input string at all possible positions.

For example, take "abc" as the original string:

1. Start with a blank `list()` object. This is actually the last call of recursive function stack. Pick a character 'c' of original string, making the output as `['c']`
2. Pick next character b of original input string, and place the current character at different indices of the each sub-string of previous output. **We can make use of the sub-string of previous output, to create a new sub-string.** Now, the output will become `['bc', 'cb']`.
3. Pick next character a of original input string, and place the current character at different indices of the each sub-string of previous output. Now, the output will become `['abc', 'bac', 'bca', 'acb', 'cab', 'cba']`..

Exercise - Write the function definition here

```
def permutations(string):
    return return_permutations(string, 0)

def return_permutations(string, index):
    output = list()

    if index >= len(string):
        return [""]
```

```

    small_output = return_permutations(string, index + 1)
    current_char = string[index]

    for subString in small_output:
        for index in range(len(small_output[0]) + 1):
            new_subString = subString[0: index] + current_char +
subString[index:]
            output.append(new_subString)

    return output

def test_function(test_case):
    string = test_case[0]
    solution = test_case[1]
    output = permutations(string)

    output.sort()
    solution.sort()

    if output == solution:
        print("Pass")
    else:
        print("Fail")

string = 'ab'
solution = ['ab', 'ba']
test_case = [string, solution]
test_function(test_case)

Pass

string = 'abc'
output = ['abc', 'bac', 'bca', 'acb', 'cab', 'cba']
test_case = [string, output]
test_function(test_case)

Pass

string = 'abcd'
output = ['abcd', 'bacd', 'bcad', 'bcda', 'acbd', 'cabd', 'cbad',
'cbda', 'acdb', 'cadb', 'cdab', 'cdba', 'abdc', 'badc', 'bdac',
'bdca', 'adbc', 'dabc', 'dbac', 'dbca', 'adcb', 'dacb', 'dcab',
'dcba']
test_case = [string, output]
test_function(test_case)

Pass

def permutations(string):
    if not string:
        return [""]

```

```

first = string[0]
rest = permutations(string[1:])

result = []
for word in rest:
    for i in range(len(word) + 1):
        result.append(word[:i] + first + word[i:])

return result

string = 'abcd'
output = ['abcd', 'bacd', 'bcad', 'bcda', 'acbd', 'cabd', 'cbad',
'cbda', 'acdb', 'cadb', 'cdab', 'cdba', 'abdc', 'badc', 'bdac',
'bdca', 'adbc', 'dabc', 'dbac', 'dbca', 'adcb', 'dacb', 'dcab',
'dcba']

result = permutations(string)

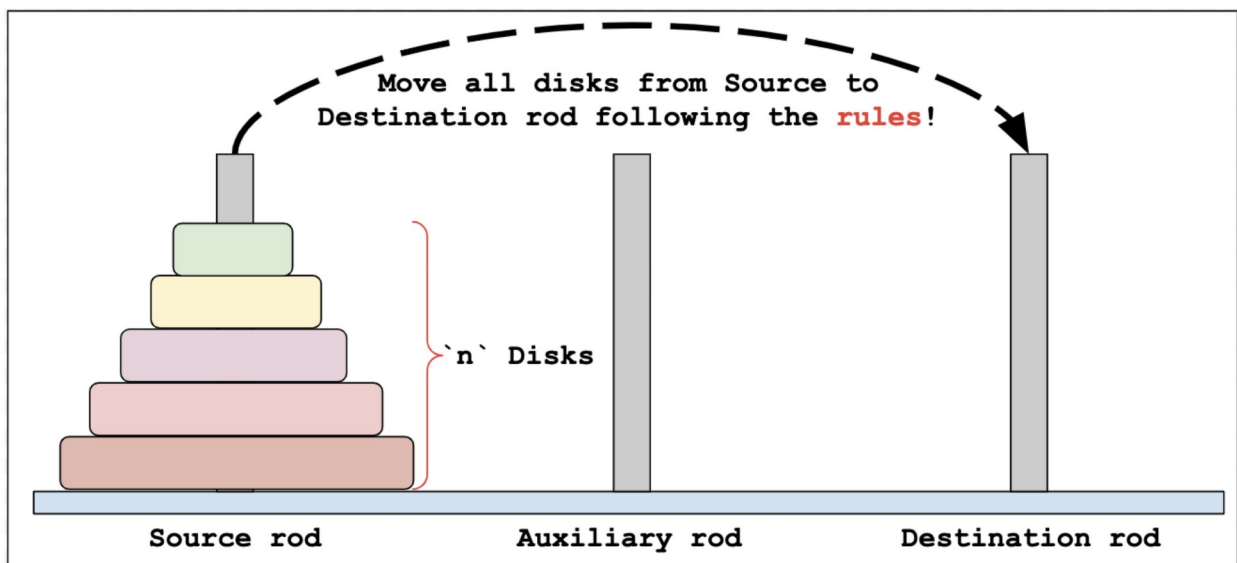
# Check correctness
print(sorted(result) == sorted(output)) # Should print: True

True

```

Keypad Combinations

A keypad on a cellphone has alphabets for all numbers between 2 and 9, as shown in the figure below:



You can make different combinations of alphabets by pressing the numbers.

For example, if you press 23, the following combinations are possible:

ad, ae, af, bd, be, bf, cd, ce, cf

Note that because 2 is pressed before 3, the first letter is always an alphabet on the number 2. Likewise, if the user types 32, the order would be

da, db, dc, ea, eb, ec, fa, fb, fc

Given an integer `num`, find out all the possible strings that can be made using digits of input `num`. Return these strings in a list. The order of strings in the list does not matter. However, as stated earlier, the order of letters in a particular string matters.

```
def get_characters(num):
    if num == 2:
        return "abc"
    elif num == 3:
        return "def"
    elif num == 4:
        return "ghi"
    elif num == 5:
        return "jkl"
    elif num == 6:
        return "mno"
    elif num == 7:
        return "pqrs"
    elif num == 8:
        return "tuv"
    elif num == 9:
        return "wxyz"
    else:
        return ""

def keypad(num):
    if num <= 1:
        return [""]
    elif 1 < num <= 9:
        return list(get_characters(num))

    last_digit = num % 10
    small_output = keypad(num // 10)
    keypad_string = get_characters(last_digit)

    output = []
    for character in keypad_string:
        for item in small_output:
            new_item = item + character
            output.append(new_item)

    return output

def get_characters(num):
    if num == 2:
```

```

        return "abc"
    elif num == 3:
        return "def"
    elif num == 4:
        return "ghi"
    elif num == 5:
        return "jkl"
    elif num == 6:
        return "mno"
    elif num == 7:
        return "pqrs"
    elif num == 8:
        return "tuv"
    elif num == 9:
        return "wxyz"
    else:
        return ""

def keypad(num, depth=0):
    indent = "|" * depth # Tree-like indentation
    print(f"{indent}→ Processing number: {num}")

    if num <= 1:
        print(f"{indent}✓ Base case reached (num <= 1), returning [''])")
        return [""]

    if 1 < num <= 9:
        chars = list(get_characters(num))
        print(f"{indent}✓ Single digit {num} maps to characters {chars}")
        return chars

    last_digit = num % 10
    remaining_num = num // 10

    print(f"{indent}→ Last digit: {last_digit}, Remaining number: {remaining_num}")
    print(f"{indent}↳ Recursively processing remaining number: {remaining_num}")

    small_output = keypad(remaining_num, depth + 1)
    keypad_string = get_characters(last_digit)
    print(f"{indent}↳ Characters for digit {last_digit}: '{keypad_string}'")

    output = []
    print(f"{indent}↳ Combining each of {small_output} with each character in '{keypad_string}'")

```

```

    for prefix in small_output:
        for char in keypad_string:
            combined = prefix + char
            print(f"{indent} • {prefix} + {char} → {combined}")
            output.append(combined)

    print(f"{indent}✓ Result for {num}: {output}")
    return output

def test_keypad(input, expected_output):
    if sorted(keypad(input)) == expected_output:
        print("Yay. We got it right.")
    else:
        print("Oops! That was incorrect.")

# Base case: list with empty string
input = 0
expected_output = [""]
test_keypad(input, expected_output)

→ Processing number: 0
✓ Base case reached (num <= 1), returning ['']
Yay. We got it right.

# Example case
input = 23
expected_output = sorted(["ad", "ae", "af", "bd", "be", "bf", "cd",
"ce", "cf"])
test_keypad(input, expected_output)

→ Processing number: 23
→ Last digit: 3, Remaining number: 2
↳ Recursively processing remaining number: 2
|   → Processing number: 2
|   ✓ Single digit 2 maps to characters ['a', 'b', 'c']
↳ Characters for digit 3: 'def'
↳ Combining each of ['a', 'b', 'c'] with each character in 'def'
    • a + d → ad
    • a + e → ae
    • a + f → af
    • b + d → bd
    • b + e → be
    • b + f → bf
    • c + d → cd
    • c + e → ce
    • c + f → cf
✓ Result for 23: ['ad', 'ae', 'af', 'bd', 'be', 'bf', 'cd', 'ce',
'cf']
Yay. We got it right.

```



```
# Example case
input = 32
expected_output = sorted(["da", "db", "dc", "ea", "eb", "ec", "fa",
"fb", "fc"])
test_keypad(input, expected_output)
```

```
→ Processing number: 32
→ Last digit: 2, Remaining number: 3
↳ Recursively processing remaining number: 3
|   → Processing number: 3
|   ✓ Single digit 3 maps to characters ['d', 'e', 'f']
↳ Characters for digit 2: 'abc'
↳ Combining each of ['d', 'e', 'f'] with each character in 'abc'
    • d + a → da
    • d + b → db
    • d + c → dc
    • e + a → ea
    • e + b → eb
    • e + c → ec
    • f + a → fa
    • f + b → fb
    • f + c → fc
✓ Result for 32: ['da', 'db', 'dc', 'ea', 'eb', 'ec', 'fa', 'fb',
'fc']
Yay. We got it right.
```

```
# Example case
input = 8
expected_output = sorted(["t", "u", "v"])
test_keypad(input, expected_output)
```

```
→ Processing number: 8
✓ Single digit 8 maps to characters ['t', 'u', 'v']
Yay. We got it right.
```

Problem Statement

Define a procedure, `deep_reverse`, that takes as input a list, and returns a new list that is the deep reverse of the input list.

This means it reverses all the elements in the list, and if any of those elements are lists themselves, reverses all the elements in the inner list, all the way down.

Note: The procedure must not change the input list itself.

Example Input: `[1, 2, [3, 4, 5], 4, 5]` Output: `[5, 4, [5, 4, 3], 2, 1]`

Hint

1. Begin with a blank final list to be returned.
2. Traverse the given list in the reverse order.

- If an item in the list is a list itself, call the same function.
- Otherwise, append the item to the final list.

```
def deep_reverse(arr):
    if len(arr) < 1:
        return arr

    reversed_items = []
    for item in arr[::-1]:
        if type(item) is list:
            item = deep_reverse(item)
        reversed_items.append(item)

    return reversed_items

def deep_reverse(arr, depth=0):
    indent = "  " * depth
    print(f"{indent}→ deep_reverse called with arr={arr}")

    if len(arr) == 0:
        print(f"{indent} Base case: empty list, returning []")
        print(f"{indent}← returning []")
        return []

    reversed_items = []
    for idx, item in enumerate(arr[::-1]):
        if isinstance(item, list):
            print(f"{indent} Recursing into sublist at reversed index {idx}: {item}")
            item = deep_reverse(item, depth + 1)
            print(f"{indent} Sublist reversed to: {item}")
        else:
            print(f"{indent} Processing non-list item at reversed index {idx}: {item}")
        reversed_items.append(item)

    print(f"{indent}← returning {reversed_items}")
    return reversed_items

# Example to try
arr = [1, [2, 3, [4, [5, 6]]]]
deep_reverse(arr)

→ deep_reverse called with arr=[1, [2, 3, [4, [5, 6]]]]
  Recursing into sublist at reversed index 0: [2, 3, [4, [5, 6]]]
→ deep_reverse called with arr=[2, 3, [4, [5, 6]]]
  Recursing into sublist at reversed index 0: [4, [5, 6]]
→ deep_reverse called with arr=[4, [5, 6]]
    Recursing into sublist at reversed index 0: [5, 6]
→ deep_reverse called with arr=[5, 6]
```

```

    Processing non-list item at reversed index 0: 6
    Processing non-list item at reversed index 1: 5
    ← returning [6, 5]
    Sublist reversed to: [6, 5]
    Processing non-list item at reversed index 1: 4
    ← returning [[6, 5], 4]
    Sublist reversed to: [[6, 5], 4]
    Processing non-list item at reversed index 1: 3
    Processing non-list item at reversed index 2: 2
    ← returning [[[6, 5], 4], 3, 2]
    Sublist reversed to: [[[6, 5], 4], 3, 2]
    Processing non-list item at reversed index 1: 1
    ← returning [[[[6, 5], 4], 3, 2], 1]
[[[[[6, 5], 4], 3, 2], 1]

```

```

def test_function(test_case):
    arr = test_case[0]
    solution = test_case[1]

    output = deep_reverse(arr)
    if output == solution:
        print("Pass")
    else:
        print("False")

```

```

arr = [1, 2, 3, 4, 5]
solution = [5, 4, 3, 2, 1]
test_case = [arr, solution]
test_function(test_case)

```

```

→ deep_reverse called with arr=[1, 2, 3, 4, 5]
    Processing non-list item at reversed index 0: 5
    Processing non-list item at reversed index 1: 4
    Processing non-list item at reversed index 2: 3
    Processing non-list item at reversed index 3: 2
    Processing non-list item at reversed index 4: 1
    ← returning [5, 4, 3, 2, 1]
Pass

```

```

arr = [1, 2, [3, 4, 5], 4, 5]
solution = [5, 4, [5, 4, 3], 2, 1]
test_case = [arr, solution]
test_function(test_case)

```

```

→ deep_reverse called with arr=[1, 2, [3, 4, 5], 4, 5]
    Processing non-list item at reversed index 0: 5
    Processing non-list item at reversed index 1: 4
    Recursing into sublist at reversed index 2: [3, 4, 5]
    → deep_reverse called with arr=[3, 4, 5]
        Processing non-list item at reversed index 0: 5

```

```

    Processing non-list item at reversed index 1: 4
    Processing non-list item at reversed index 2: 3
    ← returning [5, 4, 3]
    Sublist reversed to: [5, 4, 3]
    Processing non-list item at reversed index 3: 2
    Processing non-list item at reversed index 4: 1
    ← returning [5, 4, [5, 4, 3], 2, 1]
Pass

arr = [1, [2, 3, [4, [5, 6]]]]
solution = [[[6, 5], 4], 3, 2], 1]
test_case = [arr, solution]
test_function(test_case)

→ deep_reverse called with arr=[1, [2, 3, [4, [5, 6]]]]
    Recursing into sublist at reversed index 0: [2, 3, [4, [5, 6]]]
    → deep_reverse called with arr=[2, 3, [4, [5, 6]]]
        Recursing into sublist at reversed index 0: [4, [5, 6]]
        → deep_reverse called with arr=[4, [5, 6]]
            Recursing into sublist at reversed index 0: [5, 6]
            → deep_reverse called with arr=[5, 6]
                Processing non-list item at reversed index 0: 6
                Processing non-list item at reversed index 1: 5
                ← returning [6, 5]
                Sublist reversed to: [6, 5]
                Processing non-list item at reversed index 1: 4
                ← returning [[6, 5], 4]
                Sublist reversed to: [[6, 5], 4]
                Processing non-list item at reversed index 1: 3
                Processing non-list item at reversed index 2: 2
                ← returning [[[6, 5], 4], 3, 2]
                Sublist reversed to: [[[6, 5], 4], 3, 2]
                Processing non-list item at reversed index 1: 1
                ← returning [[[[6, 5], 4], 3, 2], 1]
            Pass

arr = [1, [2,3], 4, [5,6]]
solution = [ [6,5], 4, [3, 2], 1]
test_case = [arr, solution]
test_function(test_case)

Pass

```

Binary Search

Overview

Given a **sorted** list (say `arr`), and a key (say `target`). The binary search algorithm returns the index of the `target` element if it is present in the given `arr` list, else returns -1. Here is an overview of how the recursive version of binary search algorithm works:

1. Given a list with the lower bound (`start_index`) and the upper bound (`end_index`).
2. Find the center (say `mid_index`) of the list.
3. Check if the element at the center is your `target`? If yes, return the `mid_index`.
4. Check if the `target` is greater than that element at `mid_index`? If yes, call the same function with right sub-array w.r.t center i.e., updated indexes as `mid_index + 1` to `end_index`
5. Check if the `target` is less than that element at `mid_index`? If yes, call the same function with left sub-array w.r.t center i.e., updated indexes as `start_index` to `mid_index - 1`
6. Repeat the step above until you find the target or until the bounds are the same or cross (the upper bound is less than the lower bound).

```
def binary_search(arr, target):
    return binary_search_func(arr, 0, len(arr) - 1, target)

def binary_search_func(arr, start_index, end_index, target):
    if start_index > end_index:
        return -1

    mid_index = (start_index + end_index)//2

    if arr[mid_index] == target:
        return mid_index
    elif arr[mid_index] > target:
        return binary_search_func(arr, start_index, mid_index - 1,
target)
    else:
        return binary_search_func(arr, mid_index + 1, end_index,
target)

def binary_search(arr, target):
    print(f"Starting binary search for {target} in array {arr}")
    result = binary_search_func(arr, 0, len(arr) - 1, target, depth=0)
    print(f"Binary search completed. Result index: {result}")
    return result

def binary_search_func(arr, start_index, end_index, target, depth):
    indent = " " * depth
    print(f"{indent}Call: binary_search_func(arr,
start_index={start_index}, end_index={end_index}, target={target})")

    if start_index > end_index:
        print(f"{indent}Base case: start_index > end_index
({start_index} > {end_index}). Target not found.")
        print(f"{indent}Return: -1")
        return -1

    mid_index = (start_index + end_index) // 2
```

```

    print(f"{indent}Mid index: {mid_index}, Mid element:
{arr[mid_index]}")

    if arr[mid_index] == target:
        print(f"{indent}Target found at index {mid_index}")
        print(f"{indent}Return: {mid_index}")
        return mid_index
    elif arr[mid_index] > target:
        print(f"{indent}Target {target} is less than mid element
{arr[mid_index]}, searching left half")
        return binary_search_func(arr, start_index, mid_index - 1,
target, depth + 1)
    else:
        print(f"{indent}Target {target} is greater than mid element
{arr[mid_index]}, searching right half")
        return binary_search_func(arr, mid_index + 1, end_index,
target, depth + 1)

arr = [0, 1, 2, 3, 4, 5, 6, 7, 8]
print(binary_search(arr, 5))

```

Starting binary search for 5 in array [0, 1, 2, 3, 4, 5, 6, 7, 8]
Call: binary_search_func(arr, start_index=0, end_index=8, target=5)
Mid index: 4, Mid element: 4
Target 5 is greater than mid element 4, searching right half
Call: binary_search_func(arr, start_index=5, end_index=8, target=5)
Mid index: 6, Mid element: 6
Target 5 is less than mid element 6, searching left half
Call: binary_search_func(arr, start_index=5, end_index=5, target=5)
Mid index: 5, Mid element: 5
Target found at index 5
Return: 5
Binary search completed. Result index: 5
5

```

def binary_search(array, target):
    start_index = 0
    end_index = len(array) - 1

    while start_index <= end_index:
        mid_index = (start_index + end_index)//2          # integer


division in Python 3


        mid_element = array[mid_index]

        if target == mid_element:                          # we have


found the element


            return mid_index

```

```

        elif target < mid_element:           # the target
is less than mid element                    # we will only
            end_index = mid_index - 1        search in the left half

        else:                               # the target
is greater than mid element                 # we will
            start_index = mid_index + 1      search only in the right half

    return -1

def test_function(test_case):
    answer = binary_search(test_case[0], test_case[1])
    if answer == test_case[2]:
        print("Pass!")
    else:
        print("Fail!")

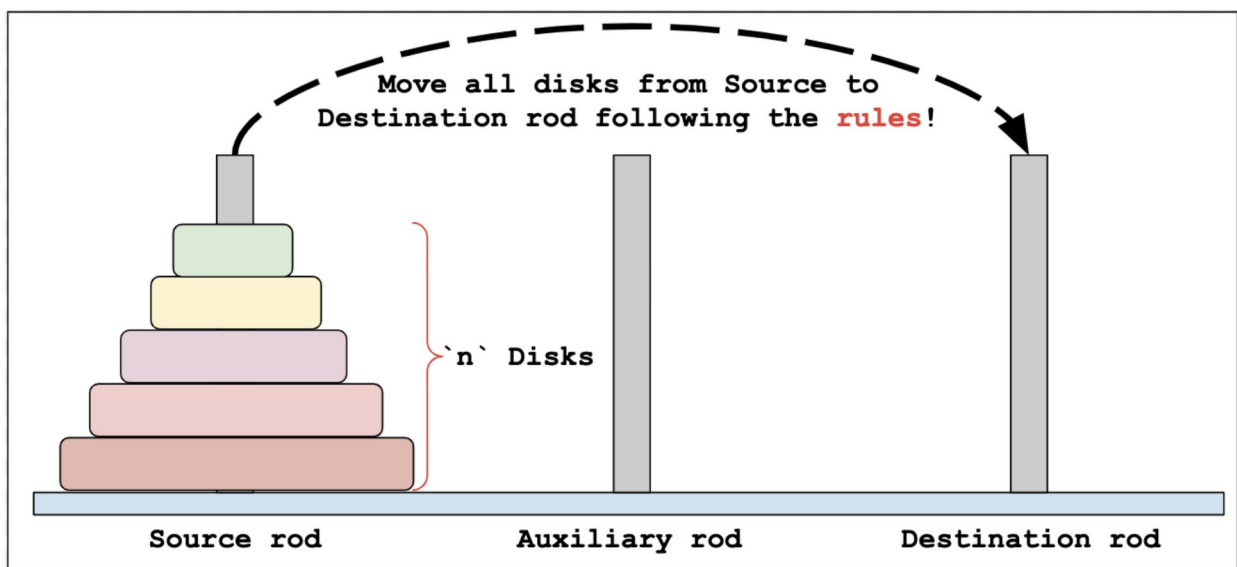
array = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
target = 6
index = 6
test_case = [array, target, index]
test_function(test_case)

Pass!

```

Problem Statement

The Tower of Hanoi is a puzzle where we have three rods and n unique sized disks. The three rods are - source, destination, and auxiliary as shown in the figure below.



Initially, all the `n` disks are present on the source rod. The final objective of the puzzle is to move all disks from the source rod to the destination rod using the auxiliary rod. **However, there are some rules applicable to all rods:** 1. Only one disk can be moved at a time. 2. A disk can be moved only if it is on the top of a rod. 3. No disk can be placed on the top of a smaller disk.

You will be given the number of disks `num_disks` as the input parameter. Write a **recursive function** `tower_of_Hanoi()` that prints the "move" steps in order to move `num_disks` number of disks from Source to Destination using the help of Auxiliary rod.

Example Illustration

For example, if you have `num_disks = 3`, then the disks should be moved as follows:

1. move disk from source to destination
2. move disk from source to auxiliary
3. move disk from destination to auxiliary
4. move disk from source to destination
5. move disk from auxiliary to source
6. move disk from auxiliary to destination
7. move disk from source to destination

You must print these steps as follows:

```
S D
S A
D A
S D
A S
A D
S D
```

Where S = source, D = destination, A = auxiliary An example illustration for `num_disks = 4` can be visualized in this [GIF from wikipedia](#)

The Idea

Assume you are writing a function that accepts the following arguments: 1. `arg1` - number of disks 2. `arg2` - rod A - this rod acts as the source (at the time of calling the function) 2. `arg3` - rod B - this rod acts as the auxiliary 2. `arg4` - rod C - this rod acts as the destination

Follow the steps below:

1. Given the `num_disks` disks on the source, along with auxiliary and destination rods

2. Check if `num_disks == 1`. This must be the termination condition, therefore use recursion to reach at this moment.
 - If yes, move disk from source to destination. (Termination condition)
 1. For `num_disks > 1`, just think of your FIRST set of steps. You want to pick the bottom most disk on the source, to be transferred to the destination. For this reason, you will will perform the steps below:
 - Step 1: Move `num_disks - 1` from source to auxiliary
 - Step 2: Now you are left with only the largest disk at source. Move the only leftover disk from source to destination
 - Step 3: You had `num_disks - 1` disks available on the auxiliary, as a result of Step 1. Move `num_disks - 1` from auxiliary to destination
-

Exercise - Write the function definition here

```
# Recursive Solution
def tower_of_Hanoi_soln(num_disks, source, auxiliary, destination):

    # Base condition
    if num_disks == 0:
        return

    # Termination condition
    if num_disks == 1:
        print("{} {}".format(source, destination))
        return

    '''Just think of one iteration, rest the Recursion will take care!'''

    # Step 1: Move `num_disks - 1` from source to auxiliary
    tower_of_Hanoi_soln(num_disks - 1, source, destination, auxiliary)

    # Step 2: Now you are left with the only largest disk at source.
    # Move the only leftover disk from source to destination
    print("{} {}".format(source, destination))

    # Step 3: Move `num_disks - 1` from auxiliary to destination
    tower_of_Hanoi_soln(num_disks - 1, auxiliary, source, destination)

def tower_of_Hanoi(num_disks):
    tower_of_Hanoi_soln(num_disks, 'S', 'A', 'D')

tower_of_Hanoi(3)

S D
S A
D A
```

```
S D
A S
A D
S D
```

Problem statement

In an encryption system where ASCII lower case letters represent numbers in the pattern `a=1`, `b=2`, `c=3`... and so on, find out all the codes that are possible for a given input number.

Example 1

- `number = 123`
- `codes_possible = ["aw", "abc", "lc"]`

Explanation: The codes are for the following number:

- `1.23 = "aw"`
- `1.2.3 = "abc"`
- `12.3 = "lc"`

Example 2

- `number = 145`
- `codes_possible = ["ade", "ne"]`

Return the codes in a list. The order of codes in the list is not important.

Note: you can assume that the input number will not contain any 0s

```
# Solution

def get_alphabet(number):
    """
    Helper function to figure out alphabet of a particular number
    Remember:
        * ASCII for lower case 'a' = 97
        * chr(num) returns ASCII character for a number e.g. chr(65)
    ==> 'A'
    """
    return chr(number + 96)

def all_codes(number):
    if number == 0:
        return [""]

    # calculation for two right-most digits e.g. if number = 1123,
    # this calculation is meant for 23
    remainder = number % 100
    output_100 = list()
```

```

if remainder <= 26 and number > 9 :

    # get all codes for the remaining number
    output_100 = all_codes(number // 100)
    alphabet = get_alphabet(remainder)

    for index, element in enumerate(output_100):
        output_100[index] = element + alphabet

    # calculation for right-most digit e.g. if number = 1123, this
    calculation is meant for 3
    remainder = number % 10

    # get all codes for the remaining number
    output_10 = all_codes(number // 10)
    alphabet = get_alphabet(remainder)

    for index, element in enumerate(output_10):
        output_10[index] = element + alphabet

    output = list()
    output.extend(output_100)
    output.extend(output_10)

    return output

def test_function(test_case):
    number = test_case[0]
    solution = test_case[1]

    output = all_codes(number)

    output.sort()
    solution.sort()

    if output == solution:
        print("Pass")
    else:
        print("Fail")

number = 123
solution = ['abc', 'aw', 'lc']
test_case = [number, solution]
test_function(test_case)

```

Pass

Markdown

Problem Statement

Given an integer array, find and return all the subsets of the array. The order of subsets in the output array is not important. However the order of elements in a particular subset should remain the same as in the input array. **Note:**

- An empty set will be represented by an empty list.
- If there are repeat integers, each occurrence must be treated as a separate entity.

Example 1

```
arr = [9, 9]

output = [[],
          [9],
          [9],
          [9, 9]]
```

Example 2

```
arr = [9, 12, 15]

output = [[],
          [15],
          [12],
          [12, 15],
          [9],
          [9, 15],
          [9, 12],
          [9, 12, 15]]
```

```
def subsets(arr): """ :param: arr - input integer array Return - list of lists (two dimensional array)
where each list represents a subset TODO: complete this method to return subsets of an array
""" pass Show Solution
```

```
def test_function(test_case): arr = test_case[0] solution = test_case[1]
```

```
output = subsets(arr)

output.sort()
solution.sort()

if output == solution:
    print("Pass")
else:
    print("Fail")
```

```
arr = [9] solution = [[], [9]] test_case = [arr, solution] test_function(test_case) Pass arr = [5, 7]
solution = [[], [7], [5], [5, 7]] test_case = [arr, solution] test_function(test_case) Pass arr = [9, 12,
```

```
15] solution = [[], [15], [12], [12, 15], [9], [9, 15], [9, 12], [9, 12, 15]] test_case = [arr, solution]
test_function(test_case) Pass arr = [9, 8, 9, 8] solution = [[], [8], [9], [9, 8], [8], [8, 8], [8, 9], [8, 9,
8], [9], [9, 8], [9, 9], [9, 9, 8], [9, 8], [9, 8, 8], [9, 8, 9], [9, 8, 9, 8]] test_case = [arr, solution]
test_function(test_case) Pass
```

```
# Solution
def subsets(arr):
    return return_subsets(arr, 0)

def return_subsets(arr, index):
    if index >= len(arr):
        return [[]]

    small_output = return_subsets(arr, index + 1)

    output = list()
    # append existing subsets
    for element in small_output:
        output.append(element)

    # add current elements to existing subsets and add them to the output
    for element in small_output:
        current = list()
        current.append(arr[index])
        current.extend(element)
        output.append(current)
    return output

def test_function(test_case):
    arr = test_case[0]
    solution = test_case[1]

    output = subsets(arr)

    output.sort()
    solution.sort()

    if output == solution:
        print("Pass")
    else:
        print("Fail")

arr = [9]
solution = [[], [9]]

test_case = [arr, solution]
test_function(test_case)

Pass
```

```
arr = [5, 7]
solution = [[], [7], [5], [5, 7]]
test_case = [arr, solution]
test_function(test_case)
```

Pass

Problem Statement

Suppose there is a staircase that you can climb in either 1 step, 2 steps, or 3 steps. In how many possible ways can you climb the staircase if the staircase has n steps? Write a recursive function to solve the problem.

Example:

- $n == 1$ then $answer = 1$
- $n == 3$ then $answer = 4$ The output is 4 because there are four ways we can climb the staircase:
 - 1 step + 1 step + 1 step
 - 1 step + 2 steps
 - 2 steps + 1 step
 - 3 steps
- $n == 5$ then $answer = 13$

```
def staircase(n):
    if n <= 0:
        return 1

    if n == 1:
        return 1
    elif n == 2:
        return 2
    elif n == 3:
        return 4

    return staircase(n - 1) + staircase(n - 2) + staircase(n - 3)

def test_function(test_case):
    n = test_case[0]
    solution = test_case[1]
    output = staircase(n)
    if output == solution:
        print("Pass")
    else:
        print("Fail")
```

```
n = 3
solution = 4
test_case = [n, solution]
test_function(test_case)
```

Pass

```
n = 4
solution = 7
test_case = [n, solution]
test_function(test_case)
```

Pass

Problem statement

Given an array `arr` and a target element `target`, find the last index of occurrence of `target` in `arr` using recursion. If `target` is not present in `arr`, return `-1`.

For example:

1. For `arr = [1, 2, 5, 5, 1, 2, 5, 4]` and `target = 5`, `output = 6`
2. For `arr = [1, 2, 5, 5, 1, 2, 5, 4]` and `target = 7`, `output = -1`

```
def last_index(arr, target):
    """
    :param: arr - input array
    :param: target - integer element
    return: int - last index of target in arr
    TODO: complete this method to find the last index of target in arr
    """
    pass

# Solution
def last_index(arr, target):
    # we start looking from the last index
    return last_index_arr(arr, target, len(arr) - 1)

def last_index_arr(arr, target, index):
    if index < 0:
        return -1

    # check if target is found
    if arr[index] == target:
        return index

    # else make a recursive call to the rest of the array
    return last_index_arr(arr, target, index - 1)
```

```
def test_function(test_case):
    arr = test_case[0]
    target = test_case[1]
    solution = test_case[2]
    output = last_index(arr, target)
    if output == solution:
        print("Pass")
    else:
        print("FAIL: Expected", solution, ", but you've got:", output)

arr = [1, 2, 5, 5, 4]
target = 5
solution = 3

test_case = [arr, target, solution]
test_function(test_case)

Pass
```

Problem statement

Given an `input_list` and a `target`, return the pair of indices in the list that holds the values which sum to the `target`. For example,

`input_list = [1, 5, 9, 7]` and `target = 8`, the answer would be `[0, 3]`

Note

1. The best solution takes $O(n)$ time. *This means that you cannot traverse the given list more than once.* **Hint - Think of an additional data structure that you should use here.**
2. You can assume that the list does not have any duplicates.

```
def pair_sum_to_target(input_list, target):
    # Create a dictionary.
    # Each element of the input_list would become a "key", and
    # the corresponding index in the input_list would become the
    "value"
    index_dict = dict()

    # Traverse through the input_list
    for index, element in enumerate(input_list):
        # `in` is the way to test for the existence of a "key" in a
        dictionary
        if (target - element) in index_dict:
            # Return the TWO indices that sum to the target
            return [index_dict[target - element], index]

        index_dict[element] = index
```



```

    return [-1,-1]                # If the target is not achieved

def pair_sum_to_target(input_list, target):
    index_dict = dict()

    for index, element in enumerate(input_list):
        complement = target - element
        print(f"\nStep {index}:")
        print(f"  Current element: {element}")
        print(f"  Complement to find: {complement}")
        print(f"  Current map: {index_dict}")

        if complement in index_dict:
            print(f"  Found! {element} + {complement} = {target}")
            print(f"  Indices: [{index_dict[complement]}, {index}]")
            return [index_dict[complement], index]

        index_dict[element] = index
        print(f"  Added {element} to map with index {index}")

    print("  No pair found.")
    return [-1, -1]

def test_function(test_case):
    output = pair_sum_to_target(test_case[0], test_case[1])
    print(output)
    if sorted(output) == test_case[2]:
        print("Pass")
    else:
        print("Fail")

test_case_1 = [[1, 5, 9, 7], 8, [0, 3]]
test_function(test_case_1)

```

Step 0:

```

Current element: 1
Complement to find: 7
Current map: {}
  Added 1 to map with index 0

```

Step 1:

```

Current element: 5
Complement to find: 3
Current map: {1: 0}
  Added 5 to map with index 1

```

Step 2:

```

Current element: 9
Complement to find: -1

```

```
Current map: {1: 0, 5: 1}
□ Added 9 to map with index 2
```

```
Step 3:
Current element: 7
Complement to find: 1
Current map: {1: 0, 5: 1, 9: 2}
□ Found! 7 + 1 = 8
Indices: [0, 3]
[0, 3]
Pass
```

```
test_case_2 = [[10, 5, 9, 8, 12, 1, 16, 6], 16, [0, 7]]
test_function(test_case_2)
```

```
Step 0:
Current element: 10
Complement to find: 6
Current map: {}
□ Added 10 to map with index 0
```

```
Step 1:
Current element: 5
Complement to find: 11
Current map: {10: 0}
□ Added 5 to map with index 1
```

```
Step 2:
Current element: 9
Complement to find: 7
Current map: {10: 0, 5: 1}
□ Added 9 to map with index 2
```

```
Step 3:
Current element: 8
Complement to find: 8
Current map: {10: 0, 5: 1, 9: 2}
□ Added 8 to map with index 3
```

```
Step 4:
Current element: 12
Complement to find: 4
Current map: {10: 0, 5: 1, 9: 2, 8: 3}
□ Added 12 to map with index 4
```

```
Step 5:
Current element: 1
Complement to find: 15
Current map: {10: 0, 5: 1, 9: 2, 8: 3, 12: 4}
```

```
□ Added 1 to map with index 5
```

Step 6:

```
Current element: 16
```

```
Complement to find: 0
```

```
Current map: {10: 0, 5: 1, 9: 2, 8: 3, 12: 4, 1: 5}
```

```
□ Added 16 to map with index 6
```

Step 7:

```
Current element: 6
```

```
Complement to find: 10
```

```
Current map: {10: 0, 5: 1, 9: 2, 8: 3, 12: 4, 1: 5, 16: 6}
```

```
□ Found! 6 + 10 = 16
```

```
Indices: [0, 7]
```

```
[0, 7]
```

```
Pass
```

```
test_case_3 = [[0, 1, 2, 3, -4], -4, [0, 4]]
```

```
test_function(test_case_3)
```

```
[0, 4]
```

```
Pass
```

Jupyter Notebook Longest-Consecutive-Subsequence Last Checkpoint: 05/07/2020 (autosaved)
Current Kernel Logo Python 3 File Edit View Insert Cell Kernel Widgets Help

Markdown

Problem Statement

Given a list of integers that contain natural numbers in random order. Write a program to find the longest possible sub sequence of consecutive numbers in the array. Return this subsequence in sorted order. *In other words, you have to return the sorted longest (sub) list of consecutive numbers present anywhere in the given list.* For example, given the list 5, 4, 7, 10, 1, 3, 55, 2, the output should be 1, 2, 3, 4, 5 **Note**

1. The solution must take $O(n)$ time. *Can you think of using a dictionary here?*
2. If two subsequences are of equal length, return the subsequence whose index of smallest element comes first. ---

The Idea:

Every element of the given `input_list` could be a part of some subsequence. Therefore, we need a way (using a dictionary) to keep track if an element has already been visited. Also, store length of a subsequence if it is maximum. For this purpose, we have to check in **forward** direction, if the `(element+1)` is available in the given dictionary, in a "while" loop. Similarly, we will check in **backward** direction for `(element-1)`, in another "while" loop. At last, if we have the smallest element and the length of the longest subsequence, we can return a **new** list starting from "smallest element" to "smallest element + length". The steps would be:

3. Create a dictionary, such that the elements of `input_list` become the "key", and the corresponding index become the "value" in the dictionary. We are creating a dictionary because the lookup time is considered to be constant in a dictionary.
4. For each `element` in the `input_list`, first mark it as visited in the dictionary
 - Check in forward direction, if the `(element+1)` is available. If yes, increment the length of subsequence
 - Check in backward direction, if the `(element - 1)` is available. If yes, increment the length of subsequence
 - Keep a track of length of longest subsequence visited so far. For the longest subsequence, store the smallest element (say `start_element`) and its index as well.
1. Return a **new** list starting from `start_element` to `start_element + length`.
 Exercise - Write the function definition here
`def longest_consecutive_subsequence(input_list):`
 pass
 Test - Let's test your function
`def test_function(test_case):`
`output = longest_consecutive_subsequence(test_case[0])`
`if output == test_case[1]:`
`print("Pass")`
`else:`
`print("Fail")`

`test_case_1 = [[5, 4, 7, 10, 1, 3, 55, 2], [1, 2, 3, 4, 5]]`
`test_function(test_case_1)`
`test_case_2 = [[2, 12, 9, 16, 10, 5, 3, 20, 25, 11, 1, 8, 6], [8, 9, 10, 11, 12]]`
`test_function(test_case_2)`
`test_case_3 = [[0, 1, 2, 3, 4], [0, 1, 2, 3, 4]]`
`test_function(test_case_3)`
 Show Solution

```

def longest_consecutive_subsequence(input_list):

    # Create a dictionary.
    # Each element of the input_list would become a "key", and
    # the corresponding index in the input_list would become the
    "value"
    element_dict = dict()

    # Traverse through the input_list, and populate the dictionary
    # Time complexity = O(n)
    for index, element in enumerate(input_list):
        element_dict[element] = index

    # Represents the length of longest subsequence
    max_length = -1

    # Represents the index of smallest element in the longest
    subsequence
    starts_at = -1

    # Traverse again - Time complexity = O(n)
    for index, element in enumerate(input_list):
  
```

```

        current_starts = index
        element_dict[element] = -1          # Mark as visited

        current_count = 1                   # length of the current
subsequence

        '''CHECK ONE ELEMENT FORWARD'''
        current = element + 1              # `current` is the expected
number

        # check if the expected number is available (as a key) in the
dictionary,
        # and it has not been visited yet (i.e., value > 0)
        # Time complexity: Constant time for checking a key and
retrieving the value = O(1)
        while current in element_dict and element_dict[current] > 0:
            current_count += 1              # increment the length of
subsequence
            element_dict[current] = -1      # Mark as visited
            current = current + 1

        '''CHECK ONE ELEMENT BACKWARD'''
        # Time complexity: Constant time for checking a key and
retrieving the value = O(1)
        current = element - 1              # `current` is the expected
number

        while current in element_dict and element_dict[current] > 0:

            current_starts = element_dict[current]          # index of
smallest element in the current subsequence
            current_count += 1                                # increment
the length of subsequence
            element_dict[current] = -1
            current = current - 1

        '''If length of current subsequence >= max length of
previously visited subsequence'''
        if current_count >= max_length:
            if current_count == max_length and current_starts >
starts_at:
                continue
            starts_at = current_starts          # index of smallest
element in the current (longest so far) subsequence
            max_length = current_count          # store the length
of current (longest so far) subsequence

        start_element = input_list[starts_at]              # smallest element

```

in the longest subsequence

```
# return a NEW list starting from `start_element` to  
`(start_element + max_length)`  
return [element for element in range(start_element, start_element  
+ max_length)]
```

```
def test_function(test_case):  
    output = longest_consecutive_subsequence(test_case[0])  
    if output == test_case[1]:  
        print("Pass")  
    else:  
        print("Fail")
```

```
test_case_1 = [[5, 4, 7, 10, 1, 3, 55, 2], [1, 2, 3, 4, 5]]  
test_function(test_case_1)
```

Pass

```
test_case_2 = [[2, 12, 9, 16, 10, 5, 3, 20, 25, 11, 1, 8, 6 ], [8, 9,  
10, 11, 12]]  
test_function(test_case_2)
```

Pass

```
test_case_3 = [[0, 1, 2, 3, 4], [0, 1, 2, 3, 4]]  
test_function(test_case_3)
```

Pass

Least Recently Used Cache We have briefly discussed caching as part of a practice problem while studying hash maps.

The lookup operation (i.e., `get()`) and `put()` / `set()` is supposed to be fast for a cache memory.

While doing the `get()` operation, if the entry is found in the cache, it is known as a cache hit. If, however, the entry is not found, it is known as a cache miss.

When designing a cache, we also place an upper bound on the size of the cache. If the cache is full and we want to add a new entry to the cache, we use some criteria to remove an element. After removing an element, we use the `put()` operation to insert the new element. The remove operation should also be fast.

For our first problem, the goal will be to design a data structure known as a Least Recently Used (LRU) cache. An LRU cache is a type of cache in which we remove the least recently used entry when the cache memory reaches its limit. For the current problem, consider both `get` and `set` operations as an use operation.

Your job is to use an appropriate data structure(s) to implement the cache.

In case of a cache hit, your get() operation should return the appropriate value. In case of a cache miss, your get() should return -1. While putting an element in the cache, your put() / set() operation must insert the element. If the cache is full, you must write code that removes the least recently used entry first and then insert the element. All operations must take O(1) time.

For the current problem, you can consider the size of cache = 5.

```
class LRU_Cache(object):

    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = {}           # stores key-value pairs
        self.order = []          # maintains usage order, most
                                # recent at the end

    def get(self, key):
        if key not in self.cache:
            return -1
        # Move accessed key to the end (most recently used)
        self.order.remove(key)
        self.order.append(key)
        return self.cache[key]

    def set(self, key, value):
        if key in self.cache:
            # Update value and usage
            self.cache[key] = value
            self.order.remove(key)
            self.order.append(key)
        else:
            if len(self.cache) >= self.capacity:
                # Remove least recently used item
                lru = self.order.pop(0)
                del self.cache[lru]
            # Add new key
            self.cache[key] = value
            self.order.append(key)

our_cache = LRU_Cache(5)

our_cache.set(1, 1)
our_cache.set(2, 2)
our_cache.set(3, 3)
our_cache.set(4, 4)

print(our_cache.get(1))      # returns 1
print(our_cache.get(2))      # returns 2
print(our_cache.get(9))      # returns -1 (not present)

our_cache.set(5, 5)
our_cache.set(6, 6)
```

```
print(our_cache.get(3))      # returns -1 (3 was least recently used  
and evicted)
```

```
1  
2  
-1  
-1
```